

MỤC LỤC

MUC LUC	1
LỜI NÓI ĐẦU	4
CHƯƠNG I: GIỚI THIỆU ĐỀ TÀI	6
1.1. Bối cảnh và lý do chọn đề tài	6
1.2. Mô tả bài toán thực tế	6
1.3. Mục tiêu hệ thống	7
1.3.1. Mục tiêu tổng quát	7
1.3.2. Mục tiêu cụ thể	7
1.4. Phạm vi triển khai	8
1.5. Phương pháp tiếp cận và định hướng thực hiện	8
CHƯƠNG 2: PHÂN TÍCH YÊU CẦU HỆ THỐNG	9
2.1. Use case và kịch bản hoạt động chính	9
2.1.1. Sơ đồ use case tổng quát	10
2.1.2. Kịch bản hoạt động chính của hệ thống	10
2.2. Yêu cầu chức năng của hệ thống	16
2.2.1. Bảng tổng hợp yêu cầu chức năng	17
2.2.2. Đặc tả các yêu cầu chức năng	17
2.3. Yêu cầu phi chức năng của hệ thống	22
2.3.1. Bảng tổng hợp yêu cầu phi chức năng	23
2.3.2. Đặc tả các yêu cầu phi chức năng	23
2.4. Các ràng buộc kỹ thuật của hệ thống	26
2.4.1. Bảng tổng hợp ràng buộc kỹ thuật	26
2.4.2. Đặc tả các ràng buộc kỹ thuật	27
2.5. Tiêu chí đánh giá thành công của mô hình	30
2.5.1. Bảng tiêu chí đánh giá thành công	30
2.5.2. Diễn giải các tiêu chí đánh giá	31
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	31
3.1. Kiến trúc tổng thể của hệ thống	31
3.2. Thiết kế phần cứng	32
3.2.1. Lựa chọn vi điều khiển ESP32	32
3.2.2. Khối đầu vào và cảm biến	33
3.2.3. Khối chấp hành và cảnh báo	33
3.2.4. Bảng pin mapping của hệ thống	34
3.2.5. Sơ đồ kết nối logic phần cứng	34
3.3. Thiết kế phần mềm	35
3.3.1. Cấu trúc phần mềm hiện tại	35
3.3.2. Kiến trúc logic phần mềm	35

3.3.3. Thiết kế máy trạng thái hữu hạn (FSM)	36
3.3.4. Biểu đồ trạng thái hệ thống	37
3.3.5. Luồng xử lý sampling và đánh giá kết quả	37
3.3.6. Thiết kế xử lý ngắt, timer và debounce	39
3.3.7. Thiết kế xử lý lỗi và safe state	39
CHƯƠNG 4. TRIỂN KHAI VÀ LẬP TRÌNH	40
4.1. Môi trường phát triển và công cụ sử dụng	40
4.1.1. Bảng công cụ và môi trường sử dụng	40
4.2. Tổ chức mã nguồn dự án	41
4.2.1. Cấu trúc mã nguồn chính	41
4.2.2. Sơ đồ phụ thuộc logic giữa các module	43
4.3. Triển khai các module chính	43
4.3.1. Module cấu hình hệ thống	43
4.3.2. Module giao tiếp phần cứng	44
4.3.3. Module điều khiển trạng thái hệ thống	44
4.3.4. Module giao diện OLED	45
4.3.5. Module telemetry và logging	45
4.4. Các kỹ thuật được áp dụng trong lập trình	46
4.4.1. Debounce nút bấm	46
4.4.2. Sampling non-blocking bằng millis()	46
4.4.3. Interrupt cho nút TEST	46
4.4.4. LEDC hardware timer cho buzzer	46
4.4.5. Fault handling nhiều mức	47
4.5. Mô phỏng hệ thống trên Wokwi	47
4.5.1. Sơ đồ triển khai mô phỏng	49
CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ	50
5.1. Mục tiêu kiểm thử	50
5.2. Môi trường và phương pháp kiểm thử	50
5.2.1. Sơ đồ quy trình kiểm thử đề xuất	52
5.3. Danh sách test case	53
5.3.1. Bảng test case đề xuất	53
5.3.2. Nhận xét về bộ test case	54
5.4. Các chỉ số đo đặc và tiêu chí đánh giá	54
5.4.1. Bảng chỉ số cần đo	54
5.4.2. Tiêu chí đánh giá đạt/chưa đạt	55
5.5. Kết quả kỳ vọng theo từng kịch bản	56
5.5.1. Kịch bản Boot / Preheat	56
5.5.2. Kịch bản PASS rồi START	56
5.5.3. Kịch bản FAIL rồi không cho START	57

5.5.4. Kịch bản SensorTimeout	57
5.7. Hạn chế của quá trình kiểm thử hiện tại	58
CHƯƠNG 6. TÍNH SÁNG TẠO, KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	58
6.1. Tính sáng tạo của đề tài	58
6.1.1. Bảng tổng hợp các điểm sáng tạo của đề tài	60
6.2. Kết quả đạt được của đề tài	60
6.2.1. Bảng tổng hợp kết quả đạt được	61
6.3. Hạn chế của hệ thống hiện tại	61
6.3.1. Bảng tổng hợp các hạn chế hiện tại	62
6.4. Hướng phát triển của đề tài	63
6.4.1. Bảng định hướng phát triển	63
6.5. Kết luận chung của đề tài	64
KẾT LUẬN	64

LỜI NÓI ĐẦU

An toàn giao thông là một trong những vấn đề được xã hội quan tâm ngày càng nhiều, đặc biệt trong bối cảnh tỉ lệ tai nạn liên quan đến việc điều khiển phương tiện sau khi sử dụng rượu bia vẫn còn ở mức đáng lo ngại. Xuất phát từ thực tế đó, nhóm lựa chọn đề tài Alcohol Interlock System — một mô hình hệ thống khóa liên động kiểm tra điều kiện an toàn trước khi cho phép khởi động — làm nội dung thực hiện cho bài tập lớn môn Xây dựng hệ thống nhúng.

Đề tài này được lựa chọn vì hai lý do chính. Thứ nhất, bài toán có tính ứng dụng rõ ràng: hệ thống phản ánh trực tiếp một nhu cầu thực tế, trong đó thiết bị phải đưa ra quyết định dựa trên dữ liệu cảm biến và chỉ cho phép thao tác tiếp theo khi điều kiện an toàn được thỏa mãn. Thứ hai, bài toán đòi hỏi phải áp dụng đồng thời nhiều nội dung cốt lõi của môn học, bao gồm thiết kế máy trạng thái, xử lý tín hiệu ADC, điều khiển ngoại vi, sử dụng ngắt phần cứng, khai thác timer và xây dựng cơ chế xử lý lỗi.

Trong quá trình thực hiện, nhóm đã thiết kế và xây dựng firmware cho vi điều khiển ESP32, tổ chức mã nguồn thành các module riêng biệt cho cấu hình, giao tiếp phần cứng, điều khiển trạng thái, giao diện OLED và telemetry. Hệ thống được mô phỏng và kiểm thử trên môi trường Wokwi local kết hợp với PlatformIO, sử dụng potentiometer thay cho cảm biến MQ3 thật để đảm bảo tính ổn định và kiểm soát được trong quá trình demo. Nhóm đã áp dụng một số kỹ thuật nhúng trong firmware như : lấy mẫu ADC non-blocking bằng `millis()`, ngắt phần cứng cho nút TEST, điều khiển buzzer bằng LEDC hardware timer và fault handling phân tầng giữa cảnh báo mềm và lỗi cứng.

Về phạm vi, đề tài được xác định ngay từ đầu là prototype mô phỏng chức năng trong môi trường học phần, chưa hướng tới thiết bị thương mại hoàn chỉnh. Việc chưa tích hợp cảm biến MQ3 thật và chưa lắp ráp kiểm thử trên phần cứng vật lý nằm trong ranh giới đã được xác định từ kế hoạch thực hiện ban đầu. Những nội dung đó sẽ là hướng phát triển tiếp theo sau khi hoàn thành bài tập lớn.

Báo cáo này trình bày toàn bộ quá trình thực hiện đề tài theo trình tự: xác định bài toán và yêu cầu hệ thống, thiết kế phần cứng và phần mềm, triển khai firmware, kiểm thử trên môi trường mô phỏng và đánh giá kết quả. Nhóm hy vọng báo cáo phản ánh trung thực những gì đã thực hiện được, những gì còn giới hạn và định hướng để tiếp tục phát triển trong tương lai.

CHƯƠNG I: GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh và lý do chọn đề tài

Trong những năm gần đây, yêu cầu đảm bảo an toàn giao thông ngày càng được quan tâm mạnh mẽ, đặc biệt đối với các hành vi điều khiển phương tiện sau khi sử dụng rượu, bia. Trên thực tế, một trong những nguyên nhân dẫn đến tai nạn giao thông nghiêm trọng là người điều khiển phương tiện vẫn cố ý hoặc vô ý vận hành xe khi không còn đủ tỉnh táo. Điều này đặt ra nhu cầu về các hệ thống hỗ trợ kiểm soát an toàn ngay từ trước thời điểm khởi động phương tiện.

Từ bài toán thực tế đó, đề tài Alcohol Interlock System được lựa chọn như một hướng tiếp cận phù hợp cho môn học Xây dựng hệ thống nhúng. Ý tưởng cốt lõi của hệ thống là: trước khi cho phép khởi động, thiết bị sẽ yêu cầu người dùng thực hiện bước kiểm tra; dựa trên dữ liệu cảm biến, vi điều khiển sẽ xử lý và quyết định có cho phép mở khóa hay không. Nếu điều kiện an toàn không được thỏa mãn, hệ thống sẽ giữ trạng thái khóa và phát cảnh báo. Đây là một bài toán có tính ứng dụng thực tiễn, đồng thời phù hợp để triển khai dưới dạng prototype nhúng vì có đầy đủ các thành phần đặc trưng như: khối đầu vào, khối xử lý, khối chấp hành, khối hiển thị và khối cảnh báo.

Ngoài ý nghĩa thực tiễn, đề tài còn phù hợp với mục tiêu học phần vì cho phép áp dụng nhiều kiến thức quan trọng của hệ thống nhúng, bao gồm: tổ chức phần mềm theo mô hình module, thiết kế máy trạng thái hữu hạn, xử lý tín hiệu số/tương tự, sử dụng ngắt phần cứng, khai thác timer phần cứng, xây dựng cơ chế xử lý lỗi và đánh giá thời gian đáp ứng của hệ thống. Trong phiên bản hiện tại, firmware đã được tổ chức lại thành nhiều module như `state_machine`, `io_devices`, `ui_oled`, `telemetry`, `config`, thay vì dồn toàn bộ logic vào một file duy nhất; đây là hướng triển khai phù hợp với yêu cầu về tính rõ ràng và khả năng bảo trì của một hệ thống nhúng.

1.2. Mô tả bài toán thực tế

Bài toán đặt ra là xây dựng một hệ thống khóa liên động kiểm tra điều kiện an toàn trước khi cho phép “khởi động” mô hình phương tiện. Trong phạm vi bài tập lớn, hệ thống không can thiệp vào xe thật mà chỉ mô phỏng quy trình logic của một thiết bị interlock. Người dùng phải nhấn nút TEST để kích hoạt quá trình đo. Vi điều khiển sau đó lấy mẫu tín hiệu cảm biến trong một khoảng thời gian xác định, tính toán kết quả trung bình, so sánh với ngưỡng đã cấu hình và đưa ra quyết định.

Nếu giá trị đo nằm dưới ngưỡng an toàn, hệ thống chuyển sang trạng thái cho phép khởi động. Khi đó, người dùng có thể nhấn nút START để kích hoạt cơ cấu chấp hành mở khóa. Ngược lại, nếu kết quả vượt ngưỡng, hệ thống duy trì trạng thái khóa, bật cảnh báo và từ chối thao tác khởi động. Toàn bộ trạng thái hoạt động được hiển thị trên màn hình OLED

và được phản hồi thêm bằng LED, buzzer để tăng tính trực quan khi demo. Hệ thống hiện tại cũng xác định rất rõ ba luồng demo chính của hệ thống là: Boot/Preheat, PASS rồi START, và FAIL rồi không cho START.

Về mặt mô phỏng, trong Wokwi hệ thống sử dụng potentiometer thay cho cảm biến MQ3 thật để tạo tín hiệu analog đầu vào, giúp quá trình demo ổn định và dễ kiểm soát hơn. Cơ cấu “khóa/mở” được mô phỏng bằng servo SG90, trong khi phần cảnh báo sử dụng buzzer và các LED đỏ, xanh, vàng. Cách triển khai này cho phép giữ đúng logic nghiệp vụ của hệ thống interlock, đồng thời giảm rủi ro do giới hạn phần cứng mô phỏng trên nền tảng web/local simulator.

1.3. Mục tiêu hệ thống

1.3.1. Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là xây dựng một prototype hệ thống nhúng kiểm soát điều kiện an toàn trước khi khởi động, trong đó vi điều khiển ESP32 đóng vai trò trung tâm xử lý, đọc dữ liệu đầu vào, đưa ra quyết định và điều khiển cơ cấu chấp hành theo đúng logic interlock.

1.3.2. Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát, hệ thống cần hoàn thành các mục tiêu cụ thể sau:

- Thiết kế và triển khai được một mô hình nhúng có khả năng kiểm tra dữ liệu đầu vào trước khi cho phép thao tác khởi động.
- Xây dựng được luồng điều khiển theo máy trạng thái hữu hạn (FSM) với các trạng thái rõ ràng như làm nóng, chờ đo, lấy mẫu, đạt điều kiện, vi phạm điều kiện, đang vận hành và lỗi hệ thống.
- Điều khiển được cơ cấu chấp hành bằng servo để mô phỏng hành vi khóa/mở.
- Cung cấp được giao diện phản hồi trực quan qua OLED, LED và buzzer.
- Tổ chức mã nguồn thành các module hợp lý để thuận tiện cho việc đọc hiểu, bảo trì và trình bày trong báo cáo.
- Bổ sung được một số kỹ thuật nâng cao ở mức phù hợp với môn học như interrupt cho nút TEST, hardware timer/LEDC cho buzzer, sampling non-blocking, và fault handling phân tầng.
- Thu thập được một số chỉ số định lượng như thời gian từ TEST đến khi ra kết quả, thời gian từ PASS đến mở khóa, độ lệch chuẩn của mẫu đo và số lần FAIL liên tiếp, nhằm phục vụ phần đánh giá và kiểm thử trong báo cáo.

1.4. Phạm vi triển khai

Trong khuôn khổ bài tập lớn môn học, hệ thống được triển khai ở mức prototype mô phỏng chức năng, chưa hướng tới thiết bị thương mại hoàn chỉnh. Phạm vi hiện tại của đề tài bao gồm:

- Sử dụng ESP32 làm vi điều khiển trung tâm.
- Sử dụng servo SG90 làm cơ cấu chấp hành chính để mô phỏng trạng thái khóa/mở.
- Sử dụng OLED SSD1306, LED và buzzer để hiển thị trạng thái và cảnh báo.
- Sử dụng hai nút nhấn TEST và START làm giao diện điều khiển trực tiếp.
- Sử dụng potentiometer thay thế MQ3 trong môi trường Wokwi nhằm giả lập tín hiệu ADC.
- Tổ chức firmware chạy theo hướng ESP32 Arduino + PlatformIO, mô phỏng bằng Wokwi local, và có thể mở rộng giám sát qua dashboard realtime như một thành phần bổ sung, không làm thay đổi logic nhúng cốt lõi.

Những nội dung không nằm trong phạm vi chính của đề tài gồm:

- can thiệp vào hệ thống điện/động cơ của xe thật;
- hiệu chuẩn pháp lý cho cảm biến nồng độ cồn;
- thiết kế sản phẩm thương mại hoàn chỉnh về cơ khí, vỏ hộp, chống nước, chống rung;
- xây dựng hệ thống cloud phức tạp hay ứng dụng di động độc lập.

Việc giới hạn phạm vi như trên là cần thiết để phù hợp với thời lượng môn học, nguồn lực thực hiện và mục tiêu đánh giá của bài tập lớn.

1.5. Phương pháp tiếp cận và định hướng thực hiện

Đề tài được thực hiện theo hướng phát triển một hệ thống nhúng theo chu trình:

xác định bài toán → phân tích yêu cầu → thiết kế phần cứng/phần mềm → triển khai firmware → mô phỏng và kiểm thử → đánh giá kết quả.

Về phần cứng, hệ thống được chia thành các khối chính: khối xử lý trung tâm, khối đầu vào, khối chấp hành và khối giao diện/cảnh báo. Về phần mềm, firmware được chia module để tách biệt nhiệm vụ cấu hình, giao tiếp phần cứng, điều khiển trạng thái, hiển thị OLED và telemetry. Entry point của chương trình chỉ thực hiện khởi tạo và gọi vòng cập nhật, trong khi phần logic nghiệp vụ được gom vào lớp điều khiển chính AlcoholInterlockController. Cách tổ chức này giúp chương trình dễ bảo trì hơn và cũng tạo thuận lợi khi mô tả kiến trúc.

Một điểm đáng chú ý trong định hướng triển khai là hệ thống không chỉ dừng ở mức “chạy được”, mà còn cố gắng thể hiện một số đặc trưng quan trọng của thiết kế nhúng tốt:

- tránh chặn luồng bằng cách dùng sampling non-blocking theo millis();
- dùng interrupt cho sự kiện nhấn nút TEST để tăng tính phản hồi;
- dùng LEDC hardware timer cho buzzer thay vì chỉ bật/tắt digital đơn thuần;
- xây dựng cơ chế warning và fault riêng cho cảm biến để phân biệt mức độ nghiêm trọng của lỗi.

Từ định hướng trên, hệ thống không chỉ phù hợp để minh họa bài toán interlock trong thực tế mà còn là một mô hình tiêu biểu cho việc áp dụng các kỹ thuật lập trình và thiết kế hệ thống nhúng trong môi trường học thuật.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU HỆ THỐNG

2.1. Use case và kịch bản hoạt động chính

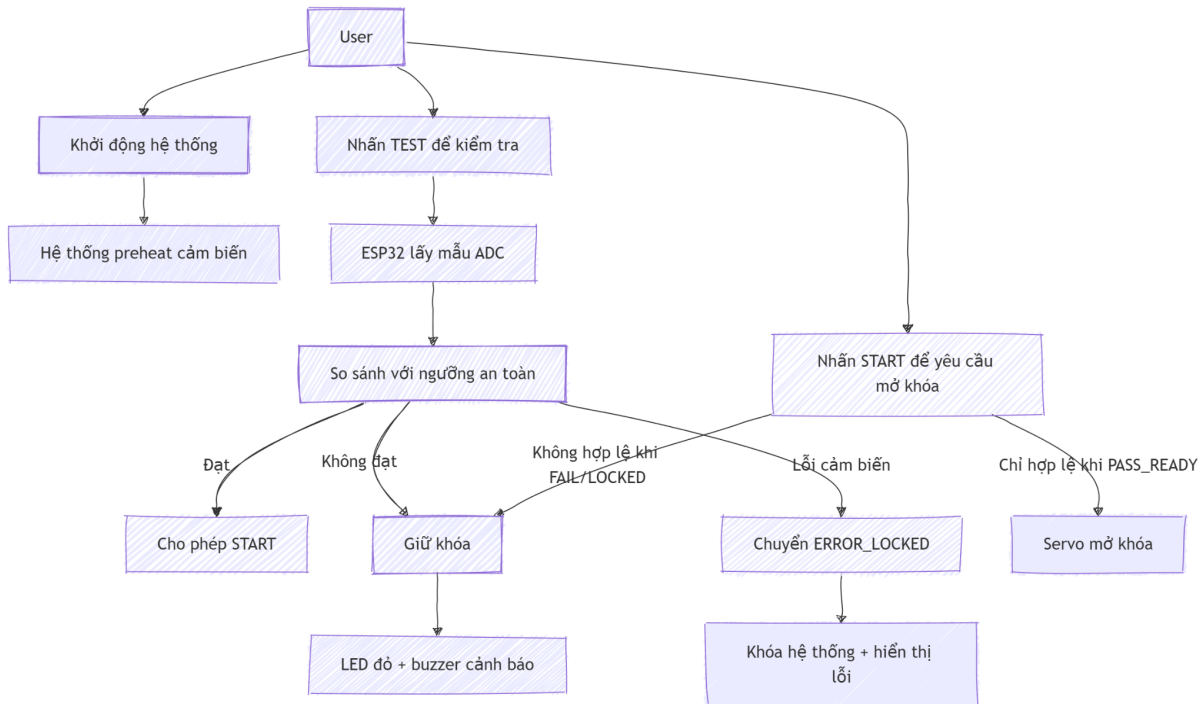
Trong phạm vi của đề tài, hệ thống có một tác nhân chính là người dùng. Người dùng tương tác với thiết bị thông qua hai nút chức năng là TEST và START, đồng thời quan sát trạng thái hệ thống qua màn hình OLED, LED và buzzer. Mục tiêu của người dùng là kiểm tra điều kiện an toàn trước khi yêu cầu mở khóa mô hình khởi động.

Về mặt chức năng, hệ thống xoay quanh ba kịch bản chính đã được xác định:

1. Boot / Preheat
2. PASS rồi START
3. FAIL rồi không cho START

Ngoài ba kịch bản chính trên, hệ thống còn có một kịch bản lỗi để phục vụ kiểm thử và đánh giá độ an toàn: khi toàn bộ cửa sổ lấy mẫu bị ghim ở vùng rail thấp hoặc rail cao, hệ thống sẽ vào trạng thái lỗi ERROR_LOCKED với fault SensorTimeout.

2.1.1. Sơ đồ use case tổng quát



Biểu đồ use case tổng quan hệ thống

2.1.2. Kịch bản hoạt động chính của hệ thống

Trong phạm vi của đề tài, hệ thống được mô hình hóa thành một số **use case chính** tương ứng với các luồng hoạt động quan trọng nhất của mô hình Alcohol Interlock. Các use case này phản ánh trực tiếp cách người dùng tương tác với thiết bị và cách hệ thống phản hồi theo từng trạng thái.

Use case 1 - Khởi động và làm nóng hệ thống

Thuộc tính	Nội dung
Tên use case	Khởi động và làm nóng hệ thống
Mã use case	UC01
Tác nhân chính	Người dùng
Mục tiêu	Khởi tạo hệ thống và đưa hệ thống vào trạng thái sẵn sàng hoạt động

Mô tả ngắn	Khi được cấp nguồn, ESP32 khởi tạo các thành phần phần cứng như OLED, servo, buzzer, LED, ADC và button, sau đó chuyển hệ thống sang trạng thái PREHEAT trong một khoảng thời gian xác định
Tiền điều kiện	Hệ thống được cấp nguồn, phần cứng được kết nối đúng
Hậu điều kiện thành công	Hệ thống hoàn tất preheat và chuyển sang trạng thái STANDBY_LOCKED, sẵn sàng nhận thao tác TEST
Hậu điều kiện thất bại	Nếu lỗi khởi tạo phần cứng quan trọng (ví dụ OLED), hệ thống chuyển sang trạng thái ERROR_LOCKED
Tác nhân phụ	ESP32, OLED, servo, buzzer, LED
Kích hoạt	Người dùng cấp nguồn cho hệ thống
Tần suất	Mỗi lần hệ thống khởi động

Luồng chính

1. Người dùng cấp nguồn cho hệ thống.
2. ESP32 thực hiện khởi tạo các module phần cứng.
3. Servo được đưa về vị trí khóa.
4. OLED hiển thị trạng thái khởi động/preheat.
5. Hệ thống duy trì trạng thái PREHEAT trong khoảng thời gian cấu hình.
6. Sau khi hết thời gian preheat, hệ thống chuyển sang STANDBY_LOCKED.

Luồng ngoại lệ

- Nếu OLED khởi tạo thất bại, hệ thống ghi nhận fault và chuyển sang ERROR_LOCKED.
- Nếu phần cứng không ổn định, hệ thống không cho phép chuyển sang trạng thái sẵn sàng.

Use case 2 - Kiểm tra điều kiện an toàn bằng nút TEST

Thuộc tính	Nội dung
Tên use case	Kiểm tra điều kiện an toàn
Mã use case	UC02
Tác nhân chính	Người dùng
Mục tiêu	Thực hiện lấy mẫu tín hiệu đầu vào và đánh giá điều kiện an toàn trước khi cho phép khởi động
Mô tả ngắn	Người dùng nhấn nút TEST, hệ thống bắt đầu lấy mẫu ADC trong một khoảng thời gian xác định, tính giá trị trung bình và so sánh với ngưỡng an toàn để đưa ra kết quả PASS hoặc FAIL
Tiền điều kiện	Hệ thống đã hoàn tất preheat và đang ở trạng thái STANDBY_LOCKED hoặc trạng thái cho phép test lại
Hậu điều kiện thành công	Hệ thống tính được kết quả đo và chuyển sang PASS_READY hoặc FAIL_LOCKED
Hậu điều kiện thất bại	Nếu phát hiện lỗi tín hiệu cảm biến nghiêm trọng, hệ thống chuyển sang ERROR_LOCKED
Tác nhân phụ	ESP32, ADC, cảm biến mô phỏng/potentiometer, OLED, buzzer, LED
Kích hoạt	Người dùng nhấn nút TEST
Tần suất	Nhiều lần trong quá trình vận hành

Luồng chính

1. Người dùng nhấn nút TEST.
2. Hệ thống nhận tín hiệu TEST.
3. Hệ thống chuyển sang trạng thái SAMPLING.
4. ESP32 lấy nhiều mẫu ADC trong cửa sổ thời gian cấu hình.
5. Firmware tính giá trị trung bình mẫu và độ lệch chuẩn.

6. Hệ thống so sánh kết quả trung bình với ngưỡng an toàn.
7. Nếu nhỏ hơn ngưỡng, hệ thống chuyển sang PASS_READY.
8. Nếu lớn hơn hoặc bằng ngưỡng, hệ thống chuyển sang FAIL_LOCKED.

Luồng ngoại lệ

- Nếu toàn bộ cửa sổ lấy mẫu bị ghim ở vùng rail thấp hoặc rail cao, hệ thống xác định lỗi cảm biến và chuyển sang ERROR_LOCKED.
- Nếu người dùng nhấn TEST khi hệ thống đang ở trạng thái không hợp lệ, thao tác có thể bị bỏ qua theo logic FSM.

Use case 3 - Cho phép khởi động khi kết quả PASS

Thuộc tính	Nội dung
Tên use case	Khởi động khi đạt điều kiện an toàn
Mã use case	UC03
Tác nhân chính	Người dùng
Mục tiêu	Cho phép người dùng mở khóa mô hình khi kết quả kiểm tra đạt yêu cầu
Mô tả ngắn	Sau khi hệ thống đánh giá PASS, người dùng nhấn nút START để yêu cầu mở khóa. Hệ thống điều khiển servo mở khóa và chuyển sang trạng thái RUNNING
Tiền điều kiện	Hệ thống đang ở trạng thái PASS_READY
Hậu điều kiện thành công	Servo mở khóa, hệ thống chuyển sang trạng thái RUNNING
Hậu điều kiện thất bại	Hệ thống không mở khóa nếu không thỏa mãn điều kiện PASS_READY
Tác nhân phụ	ESP32, servo, OLED, LED xanh, buzzer
Kích hoạt	Người dùng nhấn nút START
Tần suất	Khi người dùng đã PASS và muốn khởi động

Luồng chính

1. Hệ thống đang ở trạng thái PASS_READY.
2. Người dùng nhấn nút START.
3. Hệ thống kiểm tra điều kiện trạng thái hiện tại.
4. ESP32 điều khiển servo quay tới góc mở khóa.
5. Hệ thống bật chỉ báo trạng thái phù hợp.
6. Hệ thống chuyển sang RUNNING.

Luồng ngoại lệ

- Nếu người dùng nhấn START khi chưa PASS, hệ thống bỏ qua thao tác và không mở khóa.

Use case 4 - Từ chối khởi động khi kết quả FAIL

Thuộc tính	Nội dung
Tên use case	Từ chối khởi động khi không đạt điều kiện
Mã use case	UC04
Tác nhân chính	Người dùng
Mục tiêu	Ngăn người dùng khởi động hệ thống khi kết quả kiểm tra không đạt
Mô tả ngắn	Nếu kết quả sampling vượt ngưỡng an toàn, hệ thống chuyển sang FAIL_LOCKED, giữ servo ở trạng thái khóa, bật cảnh báo và không chấp nhận thao tác START
Tiền điều kiện	Quá trình lấy mẫu đã hoàn tất và kết quả đánh giá là FAIL
Hậu điều kiện thành công	Hệ thống tiếp tục giữ trạng thái khóa, người dùng không thể khởi động
Hậu điều kiện thất bại	Không phát sinh thao tác mở khóa ngoài ý muốn
Tác nhân phụ	ESP32, servo, buzzer, LED đỏ, OLED
Kích hoạt	Kết quả kiểm tra vượt ngưỡng an toàn
Tần suất	Khi người dùng thực hiện TEST và kết quả không đạt

Luồng chính

1. Hệ thống hoàn tất quá trình sampling.
2. Giá trị trung bình ADC lớn hơn hoặc bằng ngưỡng cấu hình.
3. Hệ thống chuyển sang FAIL_LOCKED.
4. Servo giữ nguyên vị trí khóa.
5. LED đỏ và buzzer phát cảnh báo.
6. OLED hiển thị trạng thái FAIL.
7. Nếu người dùng nhấn START, hệ thống không thực hiện mở khóa.

Luồng thay thế

- Người dùng có thể điều chỉnh đầu vào và nhấn TEST lại để thực hiện phép đo mới.

Use case UC05 – Xử lý lỗi cảm biến và đưa hệ thống về trạng thái an toàn

Thuộc tính	Nội dung
Tên use case	Xử lý lỗi cảm biến / lỗi hệ thống
Mã use case	UC05
Tác nhân chính	Hệ thống
Mục tiêu	Phát hiện lỗi nghiêm trọng và đưa hệ thống về trạng thái an toàn
Mô tả ngắn	Khi phát hiện tín hiệu cảm biến bất thường kéo dài hoặc lỗi khởi tạo phần cứng, hệ thống kích hoạt fault, chuyển sang ERROR_LOCKED và duy trì trạng thái khóa an toàn
Tiền điều kiện	Hệ thống đang hoạt động hoặc đang trong quá trình sampling

Hậu điều kiện thành công	Hệ thống chuyển sang ERROR_LOCKED, giữ khóa và hiển thị lỗi
Hậu điều kiện thất bại	Không cho phép hệ thống tiếp tục vận hành trong điều kiện không an toàn
Tác nhân phụ	ESP32, OLED, servo, buzzer, LED
Kích hoạt	Phát hiện fault như OledInitFailed hoặc SensorTimeout
Tần suất	Không thường xuyên, chỉ khi xảy ra lỗi

Luồng chính

1. Hệ thống đang hoạt động hoặc đang thực hiện sampling.
2. Firmware phát hiện điều kiện lỗi nghiêm trọng.
3. Hệ thống gọi cơ chế xử lý fault.
4. Hệ thống chuyển sang ERROR_LOCKED.
5. Servo được giữ ở trạng thái khóa.
6. OLED hiển thị thông báo lỗi.
7. Hệ thống không cho phép START cho đến khi được khởi động lại hoặc xử lý phù hợp.

Luồng ngoại lệ

- Nếu lỗi chỉ ở mức warning mềm, hệ thống có thể chỉ cảnh báo mà chưa vào fault cứng.

2.2. Yêu cầu chức năng của hệ thống

Dựa trên bài toán đã xác định và firmware hiện tại của dự án, các yêu cầu chức năng của hệ thống được tổ chức thành các nhóm chính: khởi tạo hệ thống, kiểm tra điều kiện an toàn, điều khiển trạng thái khóa/mở, hiển thị phản hồi, xử lý lỗi và ghi nhận dữ liệu phục vụ giám sát. Các yêu cầu này phản ánh trực tiếp những gì hệ thống phải thực hiện trong

quá trình vận hành thực tế. Repo hiện tại cũng đã thể hiện khá rõ các nhóm chức năng này thông qua cấu trúc module config, io_devices, state_machine, ui_oled, telemetry.

2.2.1. Bảng tổng hợp yêu cầu chức năng

Mã	Tên yêu cầu chức năng	Mô tả ngắn
FR01	Khởi tạo hệ thống	Hệ thống khởi tạo phần cứng, đưa mô hình về trạng thái an toàn ban đầu
FR02	Preheat cảm biến	Hệ thống duy trì giai đoạn preheat trước khi cho phép thao tác TEST
FR03	Nhận thao tác TEST	Người dùng có thể nhấn TEST để bắt đầu kiểm tra
FR04	Lấy mẫu ADC	Hệ thống đọc nhiều mẫu tín hiệu đầu vào trong một cửa sổ thời gian xác định
FR05	Đánh giá PASS/FAIL	Hệ thống so sánh kết quả trung bình với ngưỡng để quyết định đạt hay không đạt
FR06	Cho phép START khi PASS	Chỉ khi PASS thì START mới có hiệu lực để mở khóa
FR07	Từ chối START khi FAIL	Nếu không đạt điều kiện, hệ thống giữ khóa và không cho mở
FR08	Hiển thị trạng thái hệ thống	OLED, LED và buzzer phải phản ánh đúng trạng thái hiện tại
FR09	Xử lý lỗi hệ thống	Khi có lỗi nghiêm trọng, hệ thống chuyển sang trạng thái an toàn
FR10	Ghi log và telemetry	Hệ thống xuất log và chỉ số vận hành để phục vụ đánh giá và kiểm thử

2.2.2. Đặc tả các yêu cầu chức năng

FR01 – Khởi tạo hệ thống

Thuộc tính	Nội dung
Tên yêu cầu	Khởi tạo hệ thống

Mã yêu cầu	FR01
Mục tiêu	Bảo đảm hệ thống được thiết lập đúng trạng thái hoạt động ban đầu
Mô tả	Khi cấp nguồn, ESP32 phải khởi tạo OLED, ADC, servo, buzzer, LED và các nút bấm; sau đó đưa cơ cấu chấp hành về trạng thái khóa
Kết quả mong đợi	Hệ thống sẵn sàng chuyển sang giai đoạn preheat
Thành phần liên quan	IoDevices::begin(), state_machine.begin()

Trong quá trình triển khai, nhóm cấu hình tất cả các chân GPIO ngay tại hàm begin(). Servo được đưa về vị trí khóa đầu tiên để đảm bảo trạng thái an toàn khi mới cấp nguồn.

FR02 – Preheat cảm biến

Thuộc tính	Nội dung
Tên yêu cầu	Preheat cảm biến
Mã yêu cầu	FR02
Mục tiêu	Giữ một giai đoạn khởi động ổn định trước khi hệ thống bước vào trạng thái đo
Mô tả	Sau khi khởi tạo, hệ thống chuyển sang trạng thái PREHEAT trong một khoảng thời gian cấu hình trước
Kết quả mong đợi	Hệ thống chỉ chuyển sang STANDBY_LOCKED sau khi hết thời gian preheat
Thành phần liên quan	config::timing::kPreheatMs, updatePreheat()

Nhóm đặt thời gian preheat là 10 giây trong chế độ demo (kDemoMode = true) để vừa phản ánh đúng ý nghĩa của giai đoạn khởi động cảm biến, vừa đủ ngắn để thuận tiện khi demo.

FR03 – Nhận thao tác TEST

Thuộc tính	Nội dung
Tên yêu cầu	Nhận thao tác TEST

Mã yêu cầu	FR03
Mục tiêu	Cho phép người dùng kích hoạt quá trình kiểm tra điều kiện an toàn
Mô tả	Người dùng nhấn nút TEST, hệ thống ghi nhận sự kiện và bắt đầu chu trình lấy mẫu
Kết quả mong đợi	Hệ thống chuyển sang trạng thái SAMPLING
Thành phần liên quan	buttonPressed(ButtonId::Test, ...), interrupt onTestButtonIsr()

Ngoài polling thông thường, nhóm bổ sung interrupt phần cứng cho nút TEST. ISR chỉ đặt cờ testIrqFlag_, toàn bộ debounce và xử lý logic vẫn ở luồng chính để tránh side effect trong ngắt.

FR04 – Lấy mẫu ADC

Thuộc tính	Nội dung
Tên yêu cầu	Lấy mẫu ADC
Mã yêu cầu	FR04
Mục tiêu	Thu thập dữ liệu đầu vào đủ ổn định để phục vụ đánh giá
Mô tả	Sau khi TEST được kích hoạt, hệ thống lấy nhiều mẫu ADC trong một cửa sổ thời gian xác định
Kết quả mong đợi	Thu được đủ số mẫu cấu hình để tính toán trung bình và độ lệch chuẩn
Thành phần liên quan	kSampleCount, kSampleTotalMs, updateSampling()

Firmware lấy 20 mẫu trong khoảng 1 giây theo cơ chế non-blocking bằng millis(). Mỗi vòng update chỉ lấy tối đa một mẫu để tránh tình trạng dồn mẫu khi vòng lặp bị trễ.

FR05 – Đánh giá PASS/FAIL

Thuộc tính	Nội dung
Tên yêu cầu	Đánh giá PASS/FAIL
Mã yêu cầu	FR05

Mục tiêu	Ra quyết định an toàn dựa trên dữ liệu đã lấy mẫu
Mô tả	Hệ thống tính giá trị trung bình mẫu và so sánh với ngưỡng kAlcoholAdc
Kết quả mong đợi	Nếu nhỏ hơn ngưỡng thì PASS, ngược lại thì FAIL
Thành phần liên quan	finalizeSampling(), config::thresholds::kAlcoholAdc

Nhóm đặt ngưỡng kAlcoholAdc = 2000 phục vụ mục đích mô phỏng trên Wokwi. Giá trị này chưa phải kết quả hiệu chuẩn thực tế cho cảm biến MQ3 và sẽ cần điều chỉnh khi chuyển sang phần cứng thật.

FR06 – Cho phép START khi PASS

Thuộc tính	Nội dung
Tên yêu cầu	Cho phép START khi PASS
Mã yêu cầu	FR06
Mục tiêu	Chỉ cho phép mở khóa khi điều kiện an toàn đã đạt
Mô tả	Nếu hệ thống đang ở trạng thái PASS_READY, thao tác START sẽ được chấp nhận
Kết quả mong đợi	Servo mở khóa, hệ thống chuyển sang RUNNING
Thành phần liên quan	transitionTo(SystemState::Running)

Nhóm thiết kế nút START không có tác dụng độc lập — nó chỉ được xử lý khi hệ thống đang ở trạng thái PASS_READY, đảm bảo luồng 'kiểm tra trước, cho phép sau' không bị vượt qua.

FR07 – Từ chối START khi FAIL

Thuộc tính	Nội dung
Tên yêu cầu	Từ chối START khi FAIL
Mã yêu cầu	FR07
Mục tiêu	Giữ trạng thái khóa nếu người dùng không đạt điều kiện an toàn

Mô tả	Nếu hệ thống đang ở FAIL_LOCKED hoặc trạng thái chưa được phép, thao tác START không được mở khóa
Kết quả mong đợi	Servo vẫn giữ khóa, cảnh báo tiếp tục hoạt động theo logic tương ứng
Thành phần liên quan	FailLocked, updateFailBuzzer()

Yêu cầu này bảo đảm tính nhất quán của cơ chế interlock. Dù người dùng có thao tác START, hệ thống vẫn phải duy trì khóa nếu chưa trải qua nhánh PASS hợp lệ.

FR08 – Hiển thị trạng thái hệ thống

Thuộc tính	Nội dung
Tên yêu cầu	Hiển thị trạng thái hệ thống
Mã yêu cầu	FR08
Mục tiêu	Cung cấp phản hồi trực quan cho người dùng
Mô tả	OLED hiển thị trạng thái chi tiết; LED và buzzer phản ánh nhanh các mức pass/fail/lỗi
Kết quả mong đợi	Người dùng có thể nhận biết rõ hệ thống đang ở trạng thái nào
Thành phần liên quan	ui_oled, setIndicators(), setBuzzer()

FR09 – Xử lý lỗi hệ thống

Thuộc tính	Nội dung
Tên yêu cầu	Xử lý lỗi hệ thống
Mã yêu cầu	FR09
Mục tiêu	Đưa hệ thống về trạng thái an toàn khi xảy ra lỗi nghiêm trọng
Mô tả	Hệ thống phát hiện fault như lỗi OLED hoặc lỗi cảm biến nghiêm trọng và chuyển sang ERROR_LOCKED

Kết quả mong đợi	Hệ thống khóa an toàn, không cho START, hiển thị lỗi
Thành phần liên quan	FaultCode, enterFault(), ErrorLocked

Nhóm thiết kế hai mức phản ứng: warning mềm khi phát hiện tín hiệu gần rail trong thời gian ngắn, và fault cứng khi toàn bộ cửa sổ sampling đều bất thường.

FR10 – Ghi log và telemetry

Thuộc tính	Nội dung
Tên yêu cầu	Ghi log và telemetry
Mã yêu cầu	FR10
Mục tiêu	Hỗ trợ kiểm thử, đo đạc và giám sát vận hành
Mô tả	Hệ thống phải xuất log Serial và telemetry JSON để theo dõi trạng thái, thời gian và chỉ số vận hành
Kết quả mong đợi	Có đủ dữ liệu để kiểm thử, phân tích và trình bày kết quả
Thành phần liên quan	telemetry.cpp, AI_JSON

Các metric hiện có như testToResultMs, passReadyToUnlockMs, startToUnlockMs, sampleStdDev, consecutiveFailCount là cơ sở rất tốt cho phần kiểm thử và đánh giá ở chương sau.

2.3. Yêu cầu phi chức năng của hệ thống

Nếu yêu cầu chức năng trả lời câu hỏi “hệ thống phải làm gì”, thì yêu cầu phi chức năng trả lời câu hỏi “hệ thống phải làm như thế nào”. Đối với một bài tập lớn môn Xây dựng hệ thống nhúng, phần này đặc biệt quan trọng vì nó thể hiện tư duy kỹ thuật vượt ra ngoài việc chỉ viết code chạy được.

2.3.1. Bảng tổng hợp yêu cầu phi chức năng

Mã yêu cầu	Tên yêu cầu phi chức năng	Mô tả ngắn
------------	---------------------------	------------

NFR01	Thời gian đáp ứng	Hệ thống phải phản hồi đủ nhanh để thao tác và demo thuận lợi
NFR02	Tính an toàn	Mặc định phải ở trạng thái khóa, lỗi phải về safe state
NFR03	Tính Ổn định tín hiệu	Không kết luận từ một mẫu đơn lẻ, phải có cơ chế sampling
NFR04	Tính bảo trì	Mã nguồn phải rõ ràng, dễ đọc, dễ sửa
NFR05	Tính trực quan	Trạng thái phải dễ quan sát qua OLED, LED, buzzer
NFR06	Tính khả thi triển khai	Linh kiện phải phổ biến, dễ mô phỏng, chi phí hợp lý
NFR07	Khả năng kiểm thử	Hệ thống phải sinh được log, telemetry và metric để đánh giá

2.3.2. Đặc tả các yêu cầu phi chức năng

NFR01 – Thời gian đáp ứng

Thuộc tính	Nội dung
Tên yêu cầu	Thời gian đáp ứng
Mã yêu cầu	NFR01
Mục tiêu	Bảo đảm người dùng nhận được phản hồi đủ nhanh khi thao tác
Mô tả	Hệ thống cần có các mốc thời gian rõ ràng cho debounce, preheat, sampling, refresh UI và buzzer
Chỉ số liên quan	kButtonDebounceMs, kPreheatMs, kSampleTotalMs, kUiRefreshMs, kFailBuzzerPeriodMs

Firmware hiện tại đã định nghĩa khá rõ các tham số thời gian:

- debounce: 60 ms
- preheat: 10 giây ở demo mode
- sampling: 1 giây
- refresh UI: 200 ms
- buzzer fail: chu kỳ 500 ms.

NFR02 – Tính an toàn

Thuộc tính	Nội dung
Tên yêu cầu	Tính an toàn
Mã yêu cầu	NFR02
Mục tiêu	Đảm bảo hệ thống không mở khóa sai điều kiện
Mô tả	Mặc định hệ thống phải khóa; chỉ mở khi PASS; khi lỗi phải trở về trạng thái an toàn
Nguyên tắc	Fail-safe / safe state

Đây là yêu cầu phi chức năng quan trọng nhất của đề tài. Logic hiện tại luôn ưu tiên trạng thái khóa khi chưa có bằng chứng an toàn rõ ràng. Khi có lỗi, hệ thống đi về ERROR_LOCKED, không cho START và giữ servo ở vị trí khóa.

NFR03 – Tính Ổn định tín hiệu

Thuộc tính	Nội dung
Tên yêu cầu	Tính Ổn định tín hiệu
Mã yêu cầu	NFR03
Mục tiêu	Hạn chế kết luận sai do nhiễu hoặc dao động đầu vào
Mô tả	Hệ thống phải lấy nhiều mẫu, tính trung bình và hỗ trợ đánh giá thêm bằng độ lệch chuẩn
Cơ chế liên quan	sampledAdc, sampleStdDev, SensorWarning, SensorTimeout

Việc dùng trung bình mẫu, độ lệch chuẩn, warning mềm và fault cứng theo cửa sổ sampling giúp kết quả đầu vào Ổn định và đáng tin cậy hơn so với cách chỉ đọc một mẫu đơn lẻ.

NFR04 – Tính bảo trì

Thuộc tính	Nội dung
Tên yêu cầu	Tính bảo trì
Mã yêu cầu	NFR04

Mục tiêu	Tạo điều kiện cho việc đọc hiểu, sửa đổi và mở rộng chương trình
Mô tả	Mã nguồn phải được chia module hợp lý, tránh viết dồn toàn bộ logic vào một file
Thành phần liên quan	src/main.cpp, config, state_machine, io_devices, telemetry, ui_oled

NFR05 – Tính trực quan

Thuộc tính	Nội dung
Tên yêu cầu	Tính trực quan
Mã yêu cầu	NFR05
Mục tiêu	Giúp người dùng và giảng viên quan sát được trạng thái hệ thống nhanh chóng
Mô tả	Hệ thống phải phản hồi bằng nhiều lớp: OLED cho chi tiết, LED cho trạng thái nhanh, buzzer cho cảnh báo âm thanh
Thành phần liên quan	ui_oled, LED, buzzer

NFR06 – Tính khả thi triển khai

Thuộc tính	Nội dung
Tên yêu cầu	Tính khả thi triển khai
Mã yêu cầu	NFR06
Mục tiêu	Giữ hệ thống ở mức chi phí thấp, dễ lắp ráp, dễ mô phỏng
Mô tả	Linh kiện phải phổ biến, dễ kiếm, dễ tích hợp với ESP32 và Wokwi
Thành phần liên quan	ESP32, OLED, servo SG90, buzzer, LED, button, potentiometer

NFR07 – Khả năng kiểm thử

Thuộc tính	Nội dung
Tên yêu cầu	Khả năng kiểm thử
Mã yêu cầu	NFR07
Mục tiêu	Hỗ trợ việc đánh giá định lượng hệ thống
Mô tả	Hệ thống phải xuất được log, metric và telemetry đủ chi tiết để viết phần kiểm thử
Thành phần liên quan	telemetry.cpp, Serial log, AI_JSON

2.4. Các ràng buộc kỹ thuật của hệ thống

Ngoài yêu cầu chức năng và phi chức năng, hệ thống còn chịu các ràng buộc kỹ thuật xuất phát từ phần cứng, firmware và môi trường mô phỏng.

2.4.1. Bảng tổng hợp ràng buộc kỹ thuật

Mã ràng buộc	Tên ràng buộc	Mô tả ngắn
C01	Ràng buộc pin mapping	Các chân I/O đã được chốt đồng bộ giữa code và mô phỏng
C02	Ràng buộc ADC 12-bit	Dữ liệu đầu vào nằm trong miền 0–4095
C03	Ràng buộc mô phỏng cảm biến	Wokwi dùng potentiometer thay MQ3 thật
C04	Ràng buộc timing	Sampling, debounce, preheat và UI refresh phải theo cấu hình hiện tại
C05	Ràng buộc button model	Hệ thống hiện giả định input active-HIGH
C06	Ràng buộc nguồn và cơ cấu chấp hành	Servo cần nguồn đủ và phải dùng chung GND với ESP32 khi triển khai thật
C07	Ràng buộc ngưỡng demo	Threshold hiện tại chỉ có ý nghĩa trình bày, chưa phải calibration thực tế

C08	Ràng buộc tài nguyên vi điều khiển	Với cấu hình hiện tại, hệ thống không tiêu thụ quá 30% SRAM, đảm bảo đủ vùng nhớ cho stack, heap và các buffer I2C/Serial trong quá trình vận hành liên tục.
-----	------------------------------------	--

2.4.2. Đặc tả các ràng buộc kỹ thuật

C01 – Ràng buộc pin mapping

Thuộc tính	Nội dung
Tên ràng buộc	Pin mapping hệ thống
Mã ràng buộc	C01
Mô tả	Các chân chức năng đã được chốt cố định trong code và tài liệu hiện tại
Ảnh hưởng	Nếu đổi chân, phải cập nhật đồng bộ firmware, mô phỏng và báo cáo

Pin mapping hiện tại của hệ thống như sau:

Chức năng	GPIO
ADC / MQ3 giả lập	GPIO34
Servo SG90	GPIO15
Buzzer	GPIO13
LED đỏ	GPIO25
LED xanh	GPIO26
LED vàng	GPIO27
TEST	GPIO14
START	GPIO16
OLED SDA	GPIO21
OLED SCL	GPIO22

C02 – Ràng buộc ADC 12-bit

Thuộc tính	Nội dung
Tên ràng buộc	Miền giá trị ADC
Mã ràng buộc	C02
Mô tả	Hệ thống đọc tín hiệu analog với độ phân giải 12-bit
Giới hạn	0 đến 4095

ADC là đầu vào rất quan trọng của mô hình. Mọi đánh giá PASS/FAIL hiện đều dựa trên miền giá trị này. Vì vậy, các threshold, warning rail và fault cũng đều đang được xây dựng trên cơ sở miền 12-bit.

C03 – Ràng buộc mô phỏng cảm biến

Thuộc tính	Nội dung
Tên ràng buộc	Mô phỏng cảm biến trên Wokwi
Mã ràng buộc	C03
Mô tả	Wokwi không dùng MQ3 thật mà dùng potentiometer để giả lập đầu vào ADC
Ảnh hưởng	Giá trị threshold hiện tại chỉ mang tính demo, chưa phải hiệu chuẩn thực tế

C04 – Ràng buộc timing

Thuộc tính	Nội dung
Tên ràng buộc	Timing hệ thống
Mã ràng buộc	C04
Mô tả	Các mốc thời gian hiện tại đã được cấu hình cố định trong firmware demo
Thông số chính	Debounce 60 ms, Sampling 1 giây, Preheat 10 giây, UI refresh 200 ms

Các ràng buộc timing này quyết định hành vi thực tế của hệ thống khi demo và khi kiểm thử.

C05 – Ràng buộc button model

Thuộc tính	Nội dung
Tên ràng buộc	Mô hình nút bấm
Mã ràng buộc	C05
Mô tả	Hệ thống hiện giả định tín hiệu input active-HIGH
Ảnh hưởng	Nếu dùng nút active-LOW ngoài đời thật, cần sửa cấu hình trong firmware

C06 – Ràng buộc nguồn và servo

Thuộc tính	Nội dung
Tên ràng buộc	Nguồn cho cơ cấu chấp hành
Mã ràng buộc	C06
Mô tả	Servo cần nguồn ổn định và khi lắp thật phải dùng chung GND với ESP32
Ảnh hưởng	Nếu cấp nguồn không đủ, hệ thống có thể hoạt động không ổn định

Đây là một ràng buộc phần cứng quan trọng khi chuyển từ mô phỏng sang lắp thật.

C07 – Ràng buộc threshold demo

Thuộc tính	Nội dung
Tên ràng buộc	Ngưỡng so sánh demo
Mã ràng buộc	C07
Mô tả	kAlcoholAdc = 2000 hiện là tham số demo để phục vụ mô phỏng và trình bày
Ảnh hưởng	Khi dùng MQ3 thật phải hiệu chuẩn lại

2.5. Tiêu chí đánh giá thành công của mô hình

Sau khi xác định bài toán, yêu cầu chức năng, yêu cầu phi chức năng và các ràng buộc kỹ thuật, cần chốt lại các tiêu chí để đánh giá xem hệ thống đã đạt mục tiêu đề ra hay chưa.

2.5.1. Bảng tiêu chí đánh giá thành công

Mã tiêu chí	Tiêu chí	Điều kiện đạt
SC01	Boot / Preheat đúng	Hệ thống khởi động, preheat và tự chuyển sang STANDBY_LOCKED
SC02	PASS rồi START	Sau TEST với đầu vào an toàn, hệ thống cho phép START và servo mở khóa
SC03	FAIL rồi không cho START	Sau TEST với đầu vào vượt ngưỡng, hệ thống giữ khóa và từ chối START
SC04	Xử lý lỗi cảm biến	Khi fault SensorTimeout xảy ra, hệ thống vào ERROR_LOCKED
SC05	Hiển thị đúng trạng thái	OLED, LED, buzzer phản ánh đúng trạng thái thực tế
SC06	Có dữ liệu kiểm thử	Log và telemetry xuất ra đầy đủ metric cần thiết
SC07	Code có cấu trúc rõ	Repo được tổ chức sạch, dễ giải thích, dễ bảo trì

2.5.2. Diễn giải các tiêu chí đánh giá

Trước hết, hệ thống chỉ có thể được coi là thành công khi ba luồng vận hành chính chạy đúng như thiết kế ban đầu, gồm: Boot / Preheat, PASS rồi START, và FAIL rồi không cho START. Đây chính là ba kịch bản cốt lõi phản ánh bản chất của mô hình Alcohol Interlock. Nếu một trong ba kịch bản này không chạy đúng, hệ thống sẽ mất đi ý nghĩa nghiệp vụ quan trọng nhất.

Bên cạnh đó, hệ thống phải chứng minh được khả năng xử lý lỗi ở mức đủ nghiêm túc. Việc có fault SensorTimeout và khả năng đưa mô hình về ERROR_LOCKED là điều kiện cần để khẳng định rằng hệ thống không chỉ chạy theo “happy path”, mà còn có khả năng phản ứng khi dữ liệu cảm biến bất thường.

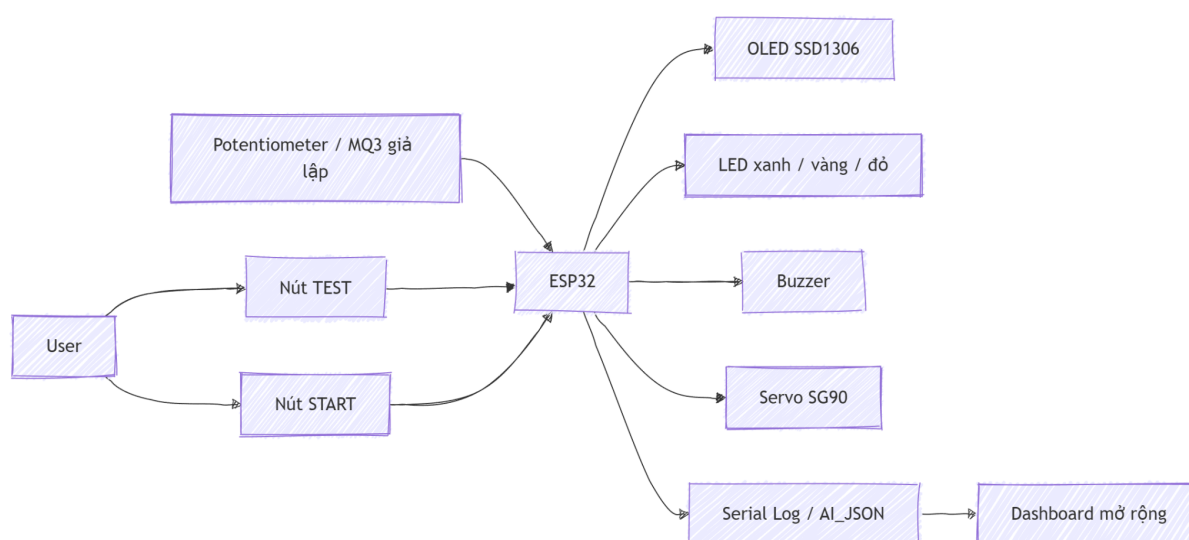
Một tiêu chí quan trọng khác là tính trực quan và khả năng kiểm thử. Hệ thống cần cho phép người quan sát dễ dàng theo dõi trạng thái qua OLED, LED và buzzer, đồng thời phải xuất được log và metric để phục vụ việc đo đạc, phân tích kết quả và viết báo cáo. Đây là điều mà phiên bản firmware hiện tại đáp ứng khá tốt thông qua module telemetry.

Cuối cùng, xét dưới góc độ bài tập lớn môn học, hệ thống còn cần đạt yêu cầu về cấu trúc phần mềm. Điều này không chỉ liên quan đến việc “code chạy được”, mà còn thể hiện ở chỗ mã nguồn được tổ chức rõ ràng, hợp lý, có thể giải thích được khi bảo vệ trước giảng viên. Với cấu trúc hệ thống và firmware hiện tại, mô hình đã có nền tảng khá tốt để đáp ứng tiêu chí này.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể của hệ thống

Ở mức tổng quát, hệ thống được thiết kế như một mô hình nhúng khép kín, trong đó ESP32 đóng vai trò trung tâm điều khiển. Mọi tín hiệu từ người dùng và đầu vào analog đều được đưa về ESP32 để xử lý; sau đó kết quả được phản hồi ra OLED, LED, buzzer và servo. Ngoài ra, firmware còn có lớp telemetry để ghi log Serial và xuất dữ liệu AI_JSON, phục vụ giám sát hoặc dashboard mở rộng trong quá trình kiểm thử. Kiến trúc này giúp hệ thống giữ được sự đơn giản cần thiết cho bài tập lớn, nhưng vẫn thể hiện được đầy đủ các khối đặc trưng của một hệ thống nhúng hoàn chỉnh.



Sơ đồ khối tổng thể hệ thống

Sơ đồ trên cho thấy hệ thống được tổ chức theo một luồng rất rõ: người dùng và đầu vào analog đi vào ESP32, còn trạng thái xử lý được phản hồi ra các thiết bị hiển thị và chấp hành. Với kiến trúc này, mọi quyết định về khóa/mở điều tập trung tại ESP32, không phân tán ra các khối ngoại vi.

3.2. Thiết kế phần cứng

Thiết kế phần cứng của hệ thống được xây dựng theo hướng tối giản nhưng đủ để phản ánh đúng bản chất của mô hình Alcohol Interlock. Các linh kiện được lựa chọn đều là những phần tử phổ biến, dễ kiếm, dễ mô phỏng trên Wokwi và dễ tích hợp với ESP32. Trong giai đoạn hiện tại, hệ thống chưa hướng tới một thiết bị thương mại hoàn chỉnh mà mới dừng ở mức prototype học phần, vì vậy phần cứng được ưu tiên theo tiêu chí: đơn giản, dễ trình bày, chi phí thấp và đủ để chứng minh logic điều khiển.

3.2.1. Lựa chọn vi điều khiển ESP32

Vi điều khiển trung tâm của hệ thống là ESP32 DevKit. Nhóm chọn ESP32 DevKit làm vi điều khiển trung tâm vì hệ thống cần đồng thời nhiều khả năng: đọc ADC, điều khiển servo, giao tiếp I2C với OLED, xử lý interrupt và hỗ trợ PWM/timer phần cứng cho buzzer. Hệ thống cần đồng thời nhiều khả năng: đọc tín hiệu analog, điều khiển servo, điều khiển buzzer bằng PWM/timer phần cứng, giao tiếp I2C với OLED, xử lý nút bấm, hỗ trợ interrupt và tổ chức firmware theo kiểu cập nhật trạng thái liên tục. Hệ thống hiện tại cũng đang được cấu hình build theo board = esp32dev và framework = arduino, cho thấy toàn bộ hệ thống đã được chốt thống nhất theo nền tảng ESP32 Arduino + PlatformIO.

ESP32 phù hợp với yêu cầu đề tài ở ba điểm chính. Thứ nhất, ESP32 đủ mạnh để vừa xử lý logic trạng thái, vừa điều khiển nhiều ngoại vi mà không làm chương trình trở nên quá nặng. Thứ hai, ESP32 có sẵn các chân ADC, I2C, GPIO và các cơ chế PWM/timer phần cứng, rất phù hợp với cấu hình hiện tại của mô hình. Thứ ba, ESP32 được Wokwi hỗ trợ tốt, giúp việc mô phỏng và kiểm thử thuận tiện hơn so với nhiều lựa chọn khác. Trong bối cảnh đồ án môn học, đây là một lựa chọn cân bằng tốt giữa tính thực tiễn và tính khả thi triển khai.

3.2.2. Khối đầu vào và cảm biến

Khối đầu vào của hệ thống gồm hai nhóm chính: đầu vào thao tác và đầu vào tín hiệu analog.

Nhóm đầu vào thao tác bao gồm hai nút:

- TEST: dùng để bắt đầu quá trình kiểm tra
- START: dùng để yêu cầu mở khóa khi hệ thống đã PASS

Theo cấu hình hiện tại, các input này được thiết kế theo mô hình active-HIGH. Trong môi trường Wokwi, hai nút được mô phỏng bằng pushbutton thường kết hợp điện trở kéo xuống để tái hiện hành vi của một module nút active-HIGH. Trong firmware, TEST còn được hỗ trợ thêm bằng interrupt để cải thiện độ phản hồi.

Nhóm đầu vào tín hiệu analog hiện được mô phỏng bằng potentiometer thay cho cảm biến MQ3 thật. Potentiometer tạo ra mức điện áp thay đổi liên tục đưa vào ADC của ESP32 tại GPIO34. Việc sử dụng potentiometer ở giai đoạn hiện tại giúp hệ thống giữ được đúng logic xử lý ADC, nhưng tránh được các vấn đề khó kiểm soát của cảm biến khí thật trong môi trường mô phỏng.

3.2.3. Khối chấp hành và cảnh báo

Khối chấp hành chính của hệ thống là servo SG90, được dùng để mô phỏng trạng thái khóa hoặc mở khóa. Khi hệ thống ở trạng thái an toàn và người dùng thao tác START hợp lệ, ESP32 gửi tín hiệu điều khiển để servo quay tới góc mở khóa. Khi ở các trạng thái khóa, servo luôn được đưa về góc khóa. Đây là phần tử quan trọng nhất trong việc biến quyết định phần mềm thành hành vi vật lý có thể quan sát được.

Khối cảnh báo của hệ thống gồm:

- OLED SSD1306 để hiển thị chi tiết trạng thái
- LED đỏ, xanh, vàng để phản ánh nhanh trạng thái
- buzzer để phát cảnh báo âm thanh

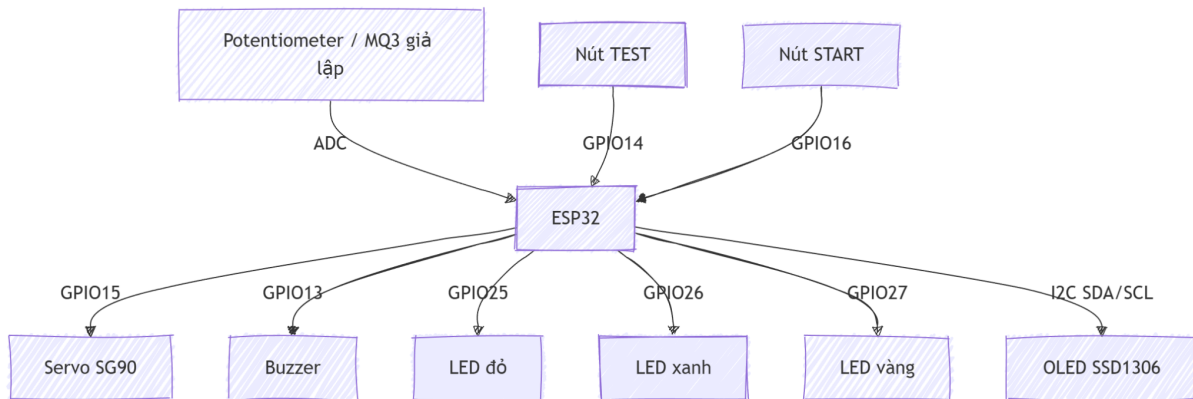
Buzzer được gắn vào LEDC hardware timer của ESP32 với tần số 2 kHz và độ phân giải 8-bit, thay vì điều khiển bằng digitalWrite đơn giản.

3.2.4. Bảng pin mapping của hệ thống

Khối chức năng	Thiết bị / tín hiệu	GPIO ESP32
Đầu vào analog	Potentiometer / MQ3 giả lập	GPIO34
Chấp hành	Servo SG90	GPIO15
Cảnh báo âm thanh	Buzzer	GPIO13
Hiển thị trạng thái	LED đỏ	GPIO25
Hiển thị trạng thái	LED xanh	GPIO26
Hiển thị trạng thái	LED vàng	GPIO27
Thao tác người dùng	TEST	GPIO14

Thao tác người dùng	START	GPIO16
Giao tiếp I2C	OLED SDA	GPIO21
Giao tiếp I2C	OLED SCL	GPIO22

3.2.5. Sơ đồ kết nối logic phần cứng



Sơ đồ kiến trúc phần cứng

3.3. Thiết kế phần mềm

Nếu phần cứng quyết định hệ thống có những thành phần gì, thì phần mềm quyết định các thành phần đó phối hợp với nhau ra sao. Trong dự án hiện tại, phần mềm được thiết kế theo hướng phân tách module, tránh kiểu viết dồn toàn bộ logic vào một file duy nhất. Hệ thống hiện tại cũng đã xác định rõ firmware source of truth nằm ở thư mục src/, trong đó mỗi nhóm chức năng chính được tách thành một module riêng.

3.3.1. Cấu trúc phần mềm hiện tại

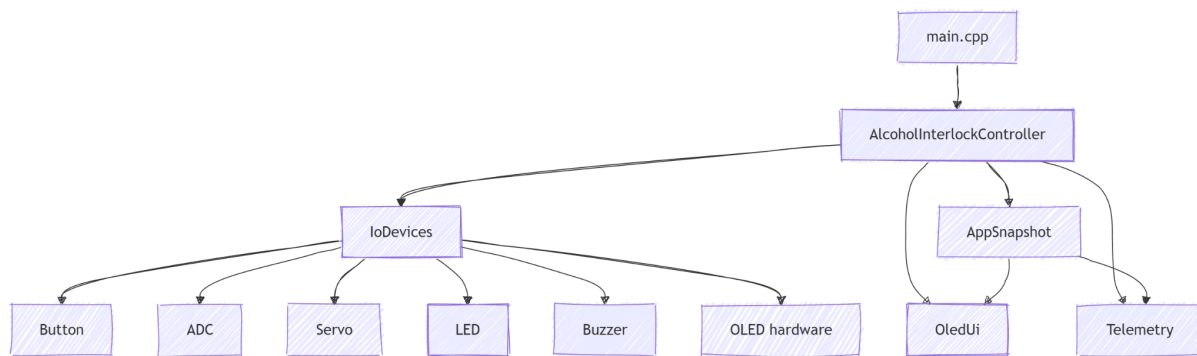
Module	Vai trò chính
main.cpp	Điểm vào chương trình, khởi tạo controller và vòng lặp cập nhật
config.h	Khai báo pin map, threshold, timing, cờ cấu hình
app_types.h	Định nghĩa state, fault, warning, snapshot và các hàm chuyển đổi trạng thái
io_devices.*	Giao tiếp phần cứng: button, ADC, servo, LED, buzzer, OLED
state_machine.*	Điều khiển nghiệp vụ chính, FSM, sampling, pass/fail, fault handling

ui_oled.*	Render giao diện OLED theo snapshot hiện tại
telemetry.*	Log Serial, metric và xuất dữ liệu AI_JSON

Với cách tổ chức này, state_machine không cần biết chi tiết điều khiển chân phần cứng, còn io_devices không chứa logic nghiệp vụ — hai lớp độc lập với nhau. Nhờ đó, hệ thống vừa dễ đọc vừa dễ sửa khi thay đổi yêu cầu hoặc thay đổi phần cứng.

Nhóm quyết định dùng vòng lặp đơn với non-blocking pattern thay vì FreeRTOS vì toàn bộ tác vụ — ADC sampling, UI refresh, telemetry, buzzer — đều có WCET ngắn và không đòi hỏi preemption. Millis() scheduling đủ đảm bảo deadline debounce 60ms và UI refresh 200ms.

3.3.2. Kiến trúc logic phần mềm



Sơ đồ kiến trúc phần mềm

Sơ đồ này cho thấy controller là trung tâm phối hợp của phần mềm. Nó đọc dữ liệu từ IoDevices, xây dựng AppSnapshot, sau đó gửi snapshot này cho OledUi và Telemetry. Đây là một cách thiết kế gọn và rõ, giúp trạng thái hệ thống được biểu diễn nhất quán ở cả lớp hiển thị lẫn lớp ghi log.

3.3.3. Thiết kế máy trạng thái hữu hạn (FSM)

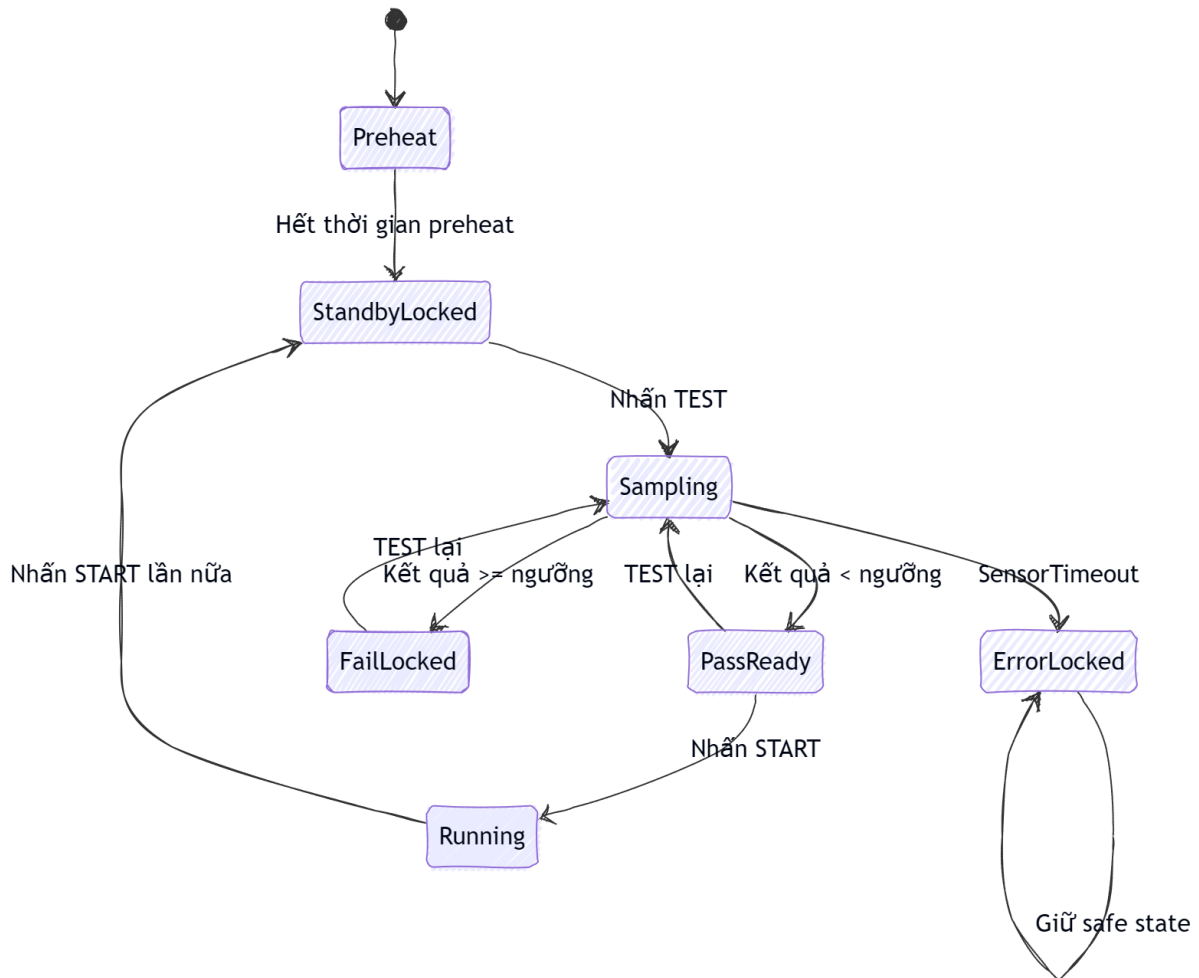
Phần mềm hiện tại của hệ thống được tổ chức quanh một máy trạng thái hữu hạn. Đây là cách tiếp cận rất phù hợp cho bài toán interlock, vì hệ thống luôn chỉ được ở một trạng thái rõ ràng tại mỗi thời điểm, và mỗi trạng thái lại có tập điều kiện chuyển tiếp riêng.

Các trạng thái hiện có gồm:

- Preheat
- StandbyLocked
- Sampling
- PassReady
- FailLocked

- Running
- ErrorLocked

3.3.4. Biểu đồ trạng thái hệ thống

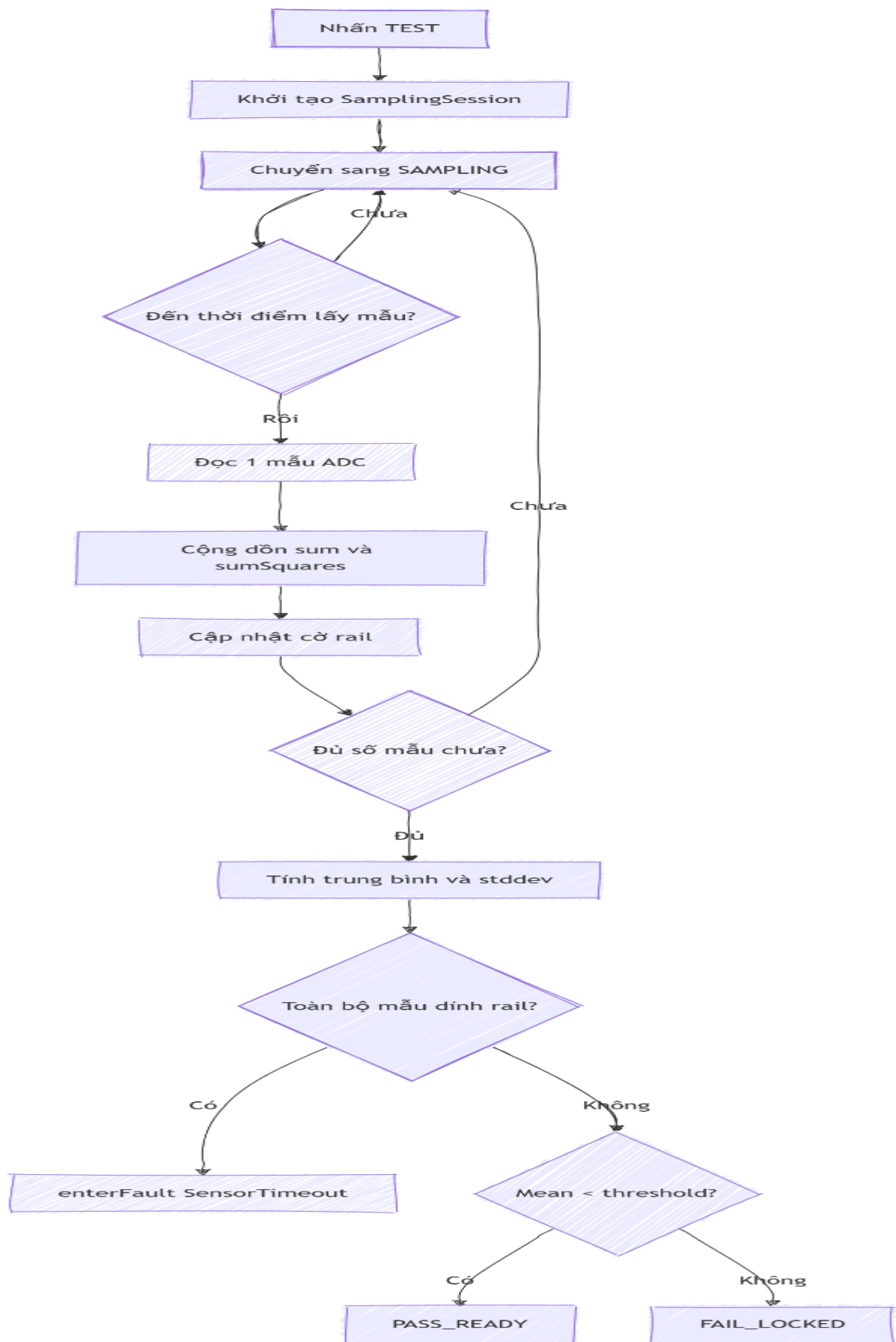


Biểu đồ trạng thái hệ thống

Biểu đồ này phản ánh trực tiếp đúng logic đang có trong `state_machine.cpp`. Nhờ mô hình FSM, các điều kiện như “START chỉ hợp lệ khi PASS” hay “lỗi phải về trạng thái khóa” được thể hiện rõ ràng và không bị lẫn lộn trong mã nguồn.

3.3.5. Luồng xử lý sampling và đánh giá kết quả

Sampling là phần cốt lõi nhất trong logic nghiệp vụ của hệ thống. Khi nhận TEST, controller khởi tạo một phiên lấy mẫu mới, lưu lại thời điểm bắt đầu, số lượng mẫu cần lấy, tổng giá trị, tổng bình phương và các cờ theo dõi tình trạng rail. Trong quá trình cập nhật, hệ thống chỉ lấy tối đa một mẫu trên mỗi vòng lặp nếu đã đến mốc thời gian cần thiết. Sau khi đủ số mẫu, hệ thống tính trung bình, phương sai, độ lệch chuẩn và ra quyết định PASS/FAIL hoặc fault.



Biểu đồ luồng xử lý sampling

3.3.6. Thiết kế xử lý ngắt, timer và debounce

Một điểm đáng chú ý trong thiết kế phần mềm của hệ thống là firmware đã sử dụng các kỹ thuật nhúng ở mức cao hơn so với một mô hình polling đơn giản.

Thứ nhất, nút TEST được gắn với một ngắt phần cứng. ISR `onTestButtonIsr()` chỉ làm một việc duy nhất là đặt cờ `testIrqFlag_ = true`, còn toàn bộ debounce và xử lý logic vẫn được thực hiện ở hàm `buttonPressed()` trong ngữ cảnh bình thường của chương trình.

Thứ hai, buzzer được thiết kế dùng LEDC hardware timer thay vì chỉ bật/tắt digital write. Ở cấu hình hiện tại, buzzer chạy ở tần số 2 kHz, độ phân giải 8-bit và duty 50% khi bật.

Thứ ba, debounce nút bấm vẫn được giữ bằng phần mềm với `kButtonDebounceMs = 60 ms`. Điều này giúp cả TEST và START đều có phản hồi ổn định, tránh hiện tượng nhấn một lần mà bị đọc thành nhiều lần do dội tiếp điểm.

3.3.7. Thiết kế xử lý lỗi và safe state

Hệ thống hiện tại được thiết kế với hai mức phản ứng khác nhau trước bất thường của tín hiệu đầu vào.

Mức thứ nhất là sensor warning. Nếu giá trị ADC bị ghim gần 0 hoặc gần 4095 trong một khoảng thời gian dài, firmware không khóa hệ thống ngay mà ghi nhận đây là một cảnh báo về tín hiệu cảm biến. Cảnh báo này có thể được xóa khi tín hiệu trở lại vùng bình thường. Cách xử lý này giúp hệ thống tránh phản ứng quá mức với các dao động hoặc trạng thái bất thường ngắn hạn.

Mức thứ hai là fault cứng. Nếu trong toàn bộ cửa sổ sampling, tất cả các mẫu đều dính rail thấp hoặc rail cao, hệ thống kết luận có lỗi nghiêm trọng và kích hoạt `SensorTimeout`. Ngoài ra, lỗi khởi tạo OLED cũng dẫn tới `OledInitFailed`. Trong cả hai trường hợp này, controller gọi `enterFault()`, đưa hệ thống sang `ErrorLocked`, khóa servo, dừng cho phép START và tiếp tục giữ thông tin lỗi qua UI/telemetry. Đây là một thiết kế fault handling rất phù hợp với một hệ thống nhúng có yêu cầu an toàn.

CHƯƠNG 4. TRIỂN KHAI VÀ LẬP TRÌNH

4.1. Môi trường phát triển và công cụ sử dụng

Để triển khai firmware cho mô hình Alcohol Interlock, đề tài sử dụng một môi trường phát triển tương đối hiện đại nhưng vẫn phù hợp với quy mô bài tập lớn. Thay vì lập trình trực tiếp bằng Arduino IDE theo kiểu đơn file, dự án được tổ chức theo hướng PlatformIO trên VS Code, từ đó cho phép quản lý source code rõ ràng hơn, build có cấu hình ổn định hơn và thuận tiện hơn trong quá trình mô phỏng, kiểm thử và bảo trì. Trong hệ thống hiện tại,

platformio.ini đã cấu hình board là esp32dev, framework là arduino, tốc độ serial monitor là 115200, đồng thời khai báo các thư viện cần thiết như Adafruit GFX Library, Adafruit SSD1306 và ESP32Servo.

Bên cạnh môi trường build local, hệ thống còn sử dụng Wokwi for VS Code để mô phỏng firmware. File wokwi.toml trong repo trỏ trực tiếp đến firmware.bin và firmware.elf được sinh ra bởi PlatformIO, nhờ đó mô phỏng luôn chạy trên đúng firmware đã build, thay vì chạy trên một bản code tách rời. Cách làm này giúp giảm đáng kể nguy cơ “code demo khác code repo”, đồng thời thuận lợi hơn cho việc kiểm thử lặp lại.

Ngoài firmware chính, source còn có một phần dashboard mở rộng dùng để hiển thị telemetry realtime. Thành phần này được viết bằng Node.js, với package.json khai báo các dependency như express, serialport, socket.io, đồng thời cung cấp các script start, mock và ports. Tuy dashboard không phải là phần cốt lõi của hệ thống nhúng, nó vẫn là một công cụ hữu ích để quan sát log và trạng thái hệ thống khi cần mở rộng quá trình kiểm thử.

4.1.1. Bảng công cụ và môi trường sử dụng

Thành phần	Công cụ / nền tảng	Vai trò
Soạn thảo và quản lý code	VS Code	Viết, chỉnh sửa và quản lý source code
Build firmware	PlatformIO	Biên dịch firmware cho ESP32
Framework lập trình	Arduino framework	Cung cấp API phần cứng và thư viện phổ biến cho ESP32
Board mục tiêu	ESP32 DevKit (esp32dev)	Vi điều khiển trung tâm của hệ thống
Mô phỏng	Wokwi local	Mô phỏng hoạt động firmware với sơ đồ mạch diagram.json

Theo dõi log	Serial Monitor	Quan sát log runtime và telemetry
Dashboard mở rộng	Node.js + Express + Socket.IO	Hiển thị trạng thái và dữ liệu realtime qua Serial

4.2. Tổ chức mã nguồn dự án

Firmware được tổ chức thành nhiều module theo chức năng thay vì dồn vào một file sketch.ino, mà đã được tách thành nhiều module theo chức năng. Hệ thống hiện tại đã chốt rất rõ rằng source of truth của firmware nằm ở thư mục src/, trong khi các file như diagram.json, platformio.ini, wokwi.toml và roadmap.md đóng vai trò hỗ trợ build, mô phỏng và vận hành dự án.

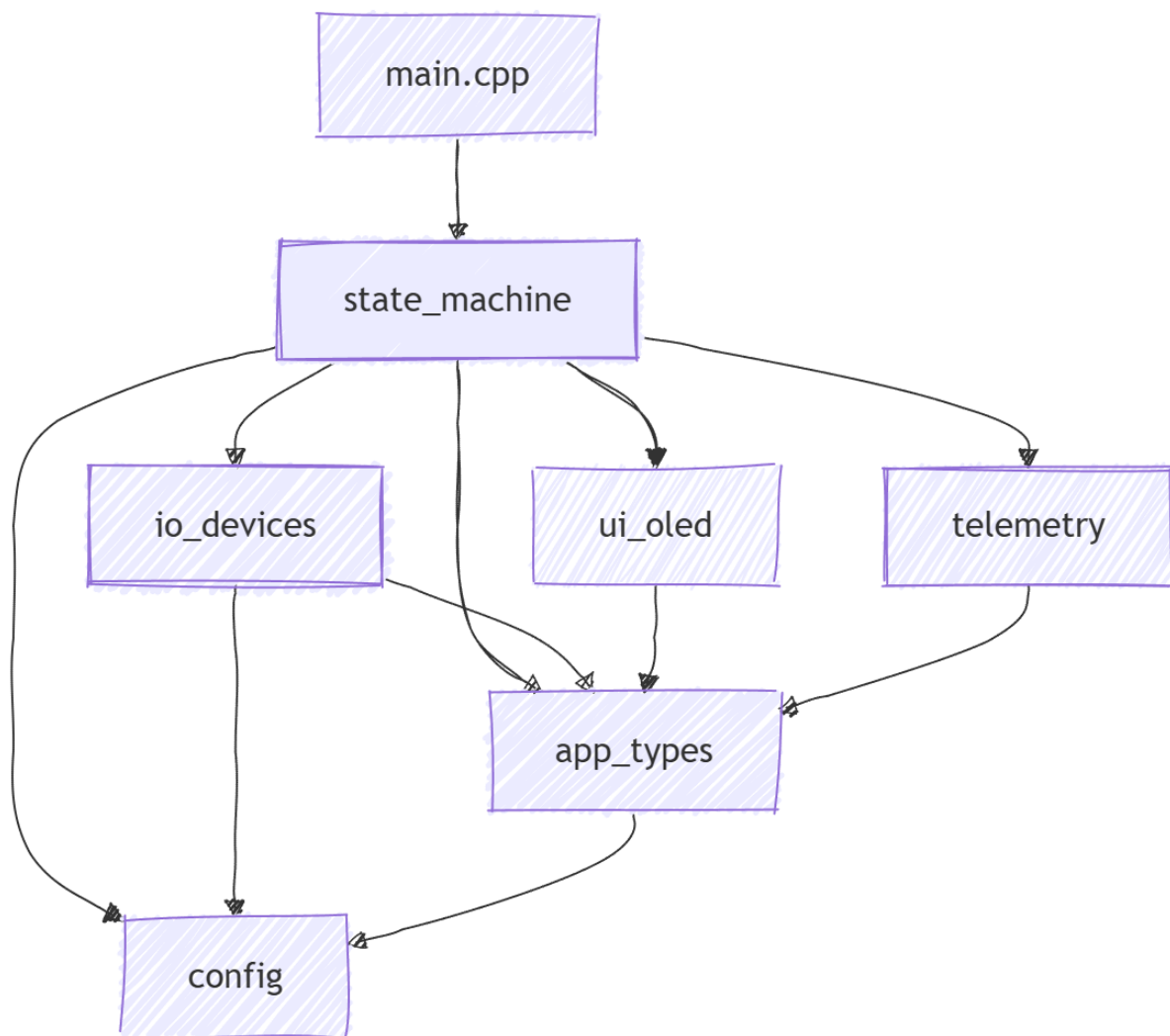
4.2.1. Cấu trúc mã nguồn chính

Đường dẫn	Vai trò
src/main.cpp	Điểm vào của firmware
src/config.h	Khai báo pin mapping, threshold, timing và các cờ cấu hình
src/app_types.h	Định nghĩa state, fault, warning, snapshot và các hàm chuyển đổi chuỗi
src/io_devices.h/.cpp	Quản lý phần cứng: button, ADC, servo, LED, buzzer, OLED
src/state_machine.h/.cpp	Chứa FSM, sampling, pass/fail, xử lý lỗi và luồng nghiệp vụ chính
src/ui_oled.h/.cpp	Hiển thị giao diện OLED theo snapshot

src/telemetry.h/.cpp	Ghi log Serial và xuất AI_JSON
diagram.json	Sơ đồ mô phỏng Wokwi
platformio.ini	Cấu hình build firmware
wokwi.toml	Kết nối firmware build local với Wokwi
roadmap.md	Hướng dẫn build, chạy mô phỏng và demo
dashboard/	Phần mở rộng realtime dashboard

Cách chia module như trên cho thấy dự án đã được refactor theo hướng đúng: phần cấu hình tách riêng, phần giao tiếp phần cứng tách riêng, phần logic nghiệp vụ tập trung trong state_machine, còn hiển thị và telemetry cũng là hai lớp độc lập. Điều này giúp code dễ theo dõi hơn rất nhiều so với cách viết dồn logic và IO vào cùng một chỗ.

4.2.2. Sơ đồ phụ thuộc logic giữa các module



Sơ đồ logic giữa các module

Sơ đồ này cho thấy state_machine là trung tâm phối hợp của firmware, trong khi io_devices, ui_oled và telemetry đóng vai trò các module dịch vụ xung quanh. config và app_types là hai lớp dùng chung, cung cấp hằng số và kiểu dữ liệu cho các phần còn lại.

4.3. Triển khai các module chính

4.3.1. Module cấu hình hệ thống

Module config.h chứa toàn bộ các hằng số cấu hình quan trọng của hệ thống, bao gồm pin mapping, kích thước OLED, góc servo, các cờ tính năng, ngưỡng ADC, các tham số timing và cấu hình button model. Việc gom các tham số này về một đầu mối giúp người phát triển có thể thay đổi cấu hình hệ thống mà không cần sửa rải rác trong nhiều file.

Trong config.h, có thể thấy hệ thống đã chốt:

- kAlcoholAdc = 2000 làm ngưỡng demo,
- kPreheatMs = 10000 khi chạy demo mode,
- kSampleCount = 20,
- kSampleTotalMs = 1000,
- kButtonDebounceMs = 60,
- kUiRefreshMs = 200,
- kSensorRailWarnMs = 8000.

Những cấu hình này phản ánh rõ hai điều. Thứ nhất, firmware hiện không dùng giá trị ngẫu nhiên, mà đã được lượng hóa bằng các tham số cụ thể. Thứ hai, code phân biệt khá rõ giữa cấu hình demo và khả năng hiệu chỉnh khi chuyển sang phần cứng thật, ví dụ như comment ở ngưỡng kAlcoholAdc hay preheat đã nêu rõ đây là tham số phục vụ Wokwi và báo cáo, không phải calibration cuối cho MQ3 thật.

4.3.2. Module giao tiếp phần cứng

io_devices là module chịu trách nhiệm giao tiếp trực tiếp với phần cứng. Module này khởi tạo LED, buzzer, button, ADC, servo và OLED; đồng thời cung cấp các hàm mức cao như buttonPressed(), readAlcoholRaw(), setBuzzer(), setIndicators(), lockVehicle(), unlockVehicle() và forceSafeState(). Thiết kế này tách biệt hoàn toàn giao tiếp phần cứng ra khỏi logic nghiệp vụ — state_machine chỉ gọi các hàm theo ý nghĩa như unlockVehicle(), setBuzzer() mà không cần biết chi tiết GPIO.

Module io_devices triển khai buzzer bằng LEDC channel riêng với tần số 2 kHz, độ phân giải 8-bit; nút TEST được gắn interrupt; ADC được constrain về miền 0–4095. Như vậy, module io_devices không chỉ là nơi gắn chân phần cứng đơn giản, mà còn là nơi thể hiện rõ các kỹ thuật nhúng cụ thể của hệ thống.

4.3.3. Module điều khiển trạng thái hệ thống

state_machine là lõi của toàn bộ firmware. Lớp AlcoholInterlockController chịu trách nhiệm điều phối các trạng thái, đọc đầu vào từ io_devices, dựng AppSnapshot, gọi ui_oled để hiển thị, đồng thời xuất log qua telemetry. Toàn bộ logic nghiệp vụ chính như preheat, bắt đầu sampling, đánh giá pass/fail, cho phép START, xử lý fail buzzer, pass beep, warning sensor và fault đều được đặt tại đây.

Một số điểm nổi bật trong module này gồm:

- FSM rõ ràng với 7 trạng thái;
- startSampling() khởi tạo phiên lấy mẫu mới;
- updateSampling() lấy tối đa 1 mẫu mỗi vòng cập nhật;

- finalizeSampling() tính mean, variance, stddev và phân loại pass/fail/fault;
- enterFault() đưa hệ thống về ErrorLocked và giữ safe state.

Cách viết hiện tại cho thấy phần nghiệp vụ đã được cô đọng khá tốt vào một controller trung tâm.

4.3.4. Module giao diện OLED

ui_oled là module chuyên trách hiển thị giao diện OLED. Thay vì để state_machine trực tiếp vẽ lên màn hình, hệ thống sử dụng một AppSnapshot làm dữ liệu đầu vào cho lớp UI. Nhờ vậy, phần hiển thị chỉ quan tâm đến việc “vẽ cái gì”, còn logic tính toán “trạng thái nào đúng” đã được xử lý sẵn ở lớp controller. Cách phân tầng này giúp tăng tính rõ ràng, tránh việc code hiển thị lẫn với code nghiệp vụ.

4.3.5. Module telemetry và logging

telemetry là module dùng để xuất log Serial và dữ liệu dạng AI_JSON. Đây là một thành phần rất có giá trị cho quá trình kiểm thử vì nó giúp chuyển trạng thái nội bộ của hệ thống thành thông tin quan sát được từ bên ngoài. Module này ghi lại:

- log boot và cấu hình nút,
- log chuyển trạng thái,
- log tiến trình sampling,
- log kết quả pass/fail,
- metric thời gian,
- sensor warning,
- fault,
- telemetry định kỳ cho dashboard.

Nhờ telemetry, đề tài không chỉ dừng ở mức “hệ thống có chạy”, mà còn có dữ liệu để chứng minh như:

- test_to_result_ms,
- pass_ready_to_unlock_ms,
- start_to_unlock_ms,
- sampleStdDev,
- consecutiveFailCount.

4.4. Các kỹ thuật được áp dụng trong lập trình

4.4.1. Debounce nút bấm

Nút bấm cơ học thường sinh ra hiện tượng dội tiếp điểm, khiến một lần nhấn có thể bị đọc thành nhiều lần nếu không xử lý đúng. Trong hệ thống hiện tại, debounce được triển khai bằng phần mềm trong hàm `buttonPressed()`, sử dụng `lastRead`, `lastStable`, `lastChangeMs` và ngưỡng thời gian `kButtonDebounceMs`. Cách làm này vừa đủ đơn giản cho bài tập lớn, vừa đủ chắc chắn để hệ thống không bị trigger sai trong quá trình demo.

4.4.2. Sampling non-blocking bằng millis()

Thay vì dùng `delay()` để chờ đủ thời gian giữa các mẫu, firmware hiện tại sử dụng mốc thời gian `nextSampleAtMs` và chỉ lấy một mẫu khi đã tới thời điểm cần thiết. Cách làm này giúp vòng lặp chính không bị khóa trong suốt quá trình sampling. Nhờ cơ chế này, UI, log và các xử lý khác vẫn tiếp tục được cập nhật trong suốt thời gian sampling mà không bị blocking.

4.4.3. Interrupt cho nút TEST

ISR `onTestButtonIsr()` chỉ đặt cờ `testIrqFlag_ = true`. Việc debounce và xử lý nhấn hợp lệ vẫn thực hiện tại hàm `buttonPressed()` trong luồng chính, tránh side effect khi thực thi nhiều logic bên trong ISR.

4.4.4. LEDC hardware timer cho buzzer

Buzzer hiện sử dụng LEDC của ESP32 với channel riêng, tần số 2 kHz và duty 50% khi bật. So với cách điều khiển bằng digital write đơn thuần, giải pháp này tạo ra âm báo đúng nghĩa, nghe rõ hơn và ổn định hơn trong lúc demo.

4.4.5. Fault handling nhiều mức

Firmware hiện phân biệt giữa:

- **warning mềm** khi ADC ở gần rail quá lâu,
- **fault cứng** như `SensorTimeout` hoặc `OledInitFailed`.

Khi gặp warning mềm, hệ thống vẫn hoạt động nhưng phát cảnh báo. Khi gặp fault cứng, controller gọi `enterFault()` và đưa toàn bộ mô hình về `ErrorLocked`. Cách phân tầng này cho thấy hệ thống đã có thiết kế fault handling tương đối có chủ đích, thay vì chỉ xử lý lỗi theo kiểu “có lỗi thì dừng” đơn giản.

4.5. Mô phỏng hệ thống trên Wokwi

Để phục vụ demo và kiểm thử, hệ thống được mô phỏng trên Wokwi local thông qua file `diagram.json`. File này mô tả đầy đủ các thành phần như:

- ESP32 DevKit,

- OLED SSD1306,
- servo SG90,
- buzzer,
- LED đỏ/xanh/vàng,
- hai nút TEST và START,
- hai điện trở kéo xuống cho nút,
- potentiometer mô phỏng đầu vào ADC.

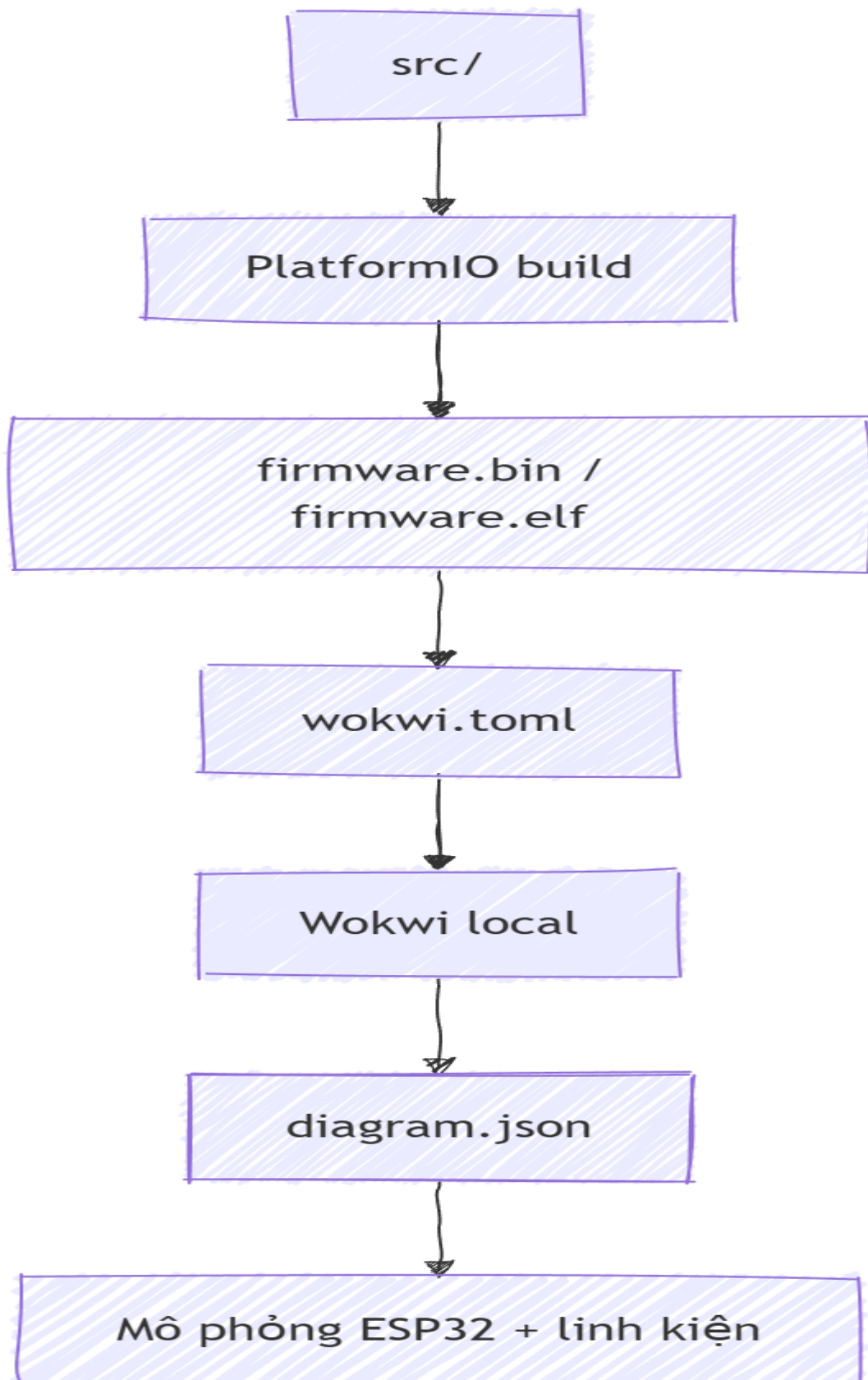
Điểm quan trọng là diagram.json đã đồng bộ với firmware ở các chân:

- TEST → D14,
- START → D16,
- ADC → D34,
- servo → D15,
- buzzer → D13,
- OLED → D21/D22,
- LED → D25/D26/D27.

Roadmap đã xác định rõ quy trình chạy mô phỏng:

1. build firmware bằng pio run,
2. mở diagram.json,
3. chạy Wokwi: Start Simulator.

4.5.1. Sơ đồ triển khai mô phỏng



Sơ đồ triển khai

CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ

5.1. Mục tiêu kiểm thử

Mục tiêu của phần kiểm thử là xác nhận rằng hệ thống Alcohol Interlock hoạt động đúng theo những gì đã được xác định trong phần yêu cầu và thiết kế. Cụ thể, quá trình kiểm thử cần làm rõ bốn nhóm vấn đề chính.

Thứ nhất, hệ thống có thực sự chạy đúng luồng nghiệp vụ cốt lõi hay không. Điều này bao gồm việc kiểm tra ba nhánh quan trọng nhất của mô hình: khởi động và preheat, kiểm tra đạt điều kiện rồi cho phép START, và kiểm tra không đạt điều kiện rồi tiếp tục giữ khóa. Đây là nền tảng để đánh giá hệ thống có thực sự phản ánh đúng logic interlock đã đặt ra hay không.

Thứ hai, kiểm thử phải xác nhận rằng các khối phần cứng và phần mềm phối hợp đúng với nhau trong từng trạng thái. Không chỉ riêng kết quả PASS/FAIL, mà các phản hồi qua OLED, LED, buzzer và servo cũng phải đồng bộ với trạng thái hệ thống. Nếu chỉ có logic nội bộ đúng nhưng đầu ra phần cứng không phản ánh đúng thì mô hình vẫn chưa thể coi là hoàn chỉnh. Phần này gắn trực tiếp với các hàm setIndicators(), setBuzzer(), lockVehicle(), unlockVehicle() và lớp OledUi.

Thứ ba, hệ thống cần được đánh giá ở góc độ thời gian và độ ổn định xử lý. Trong firmware hiện tại, một số chỉ số đã được chuẩn bị sẵn để phục vụ việc này, bao gồm thời gian từ lúc nhấn TEST đến lúc có kết quả, thời gian từ khi PASS_READY đến khi mở khóa, thời gian từ khi nhấn START đến khi ra lệnh mở servo, cùng với độ lệch chuẩn của cửa sổ mẫu. Những chỉ số này rất quan trọng vì chúng cho phép chuyển việc kiểm thử từ mức quan sát định tính sang đánh giá định lượng.

Thứ tư, phần kiểm thử còn nhằm xác nhận rằng hệ thống có khả năng xử lý bất thường và duy trì safe state. Trong bối cảnh của một mô hình interlock, đây là tiêu chí quan trọng không kém việc chạy đúng happy path. Các trường hợp như lỗi OLED hoặc fault SensorTimeout cần được kiểm tra để chứng minh rằng hệ thống biết cách từ chối vận hành khi tín hiệu hoặc phần cứng không đáng tin cậy.

5.2. Môi trường và phương pháp kiểm thử

Trong phạm vi hiện tại, môi trường kiểm thử chính của hệ thống được xác định là PlatformIO + Wokwi local trên VS Code, sử dụng firmware build từ thư mục src/, chạy trên board esp32dev, với sơ đồ mô phỏng được mô tả bằng diagram.json. Cách tổ chức này bảo đảm rằng mã nguồn dùng để kiểm thử là đúng mã nguồn của hệ thống, đồng thời hạn chế sự khác biệt giữa môi trường phát triển và môi trường demo. wokwi.toml cũng đã được

cấu hình để trở trực tiếp tới firmware.bin và firmware.elf do PlatformIO sinh ra, giúp quá trình mô phỏng nhất quán hơn.

Về phần cứng mô phỏng, hệ thống hiện sử dụng:

- ESP32 DevKit
- OLED SSD1306
- Servo SG90
- Buzzer
- LED đỏ, xanh, vàng
- Nút TEST
- Nút START
- Potentiometer thay cho MQ3 thật

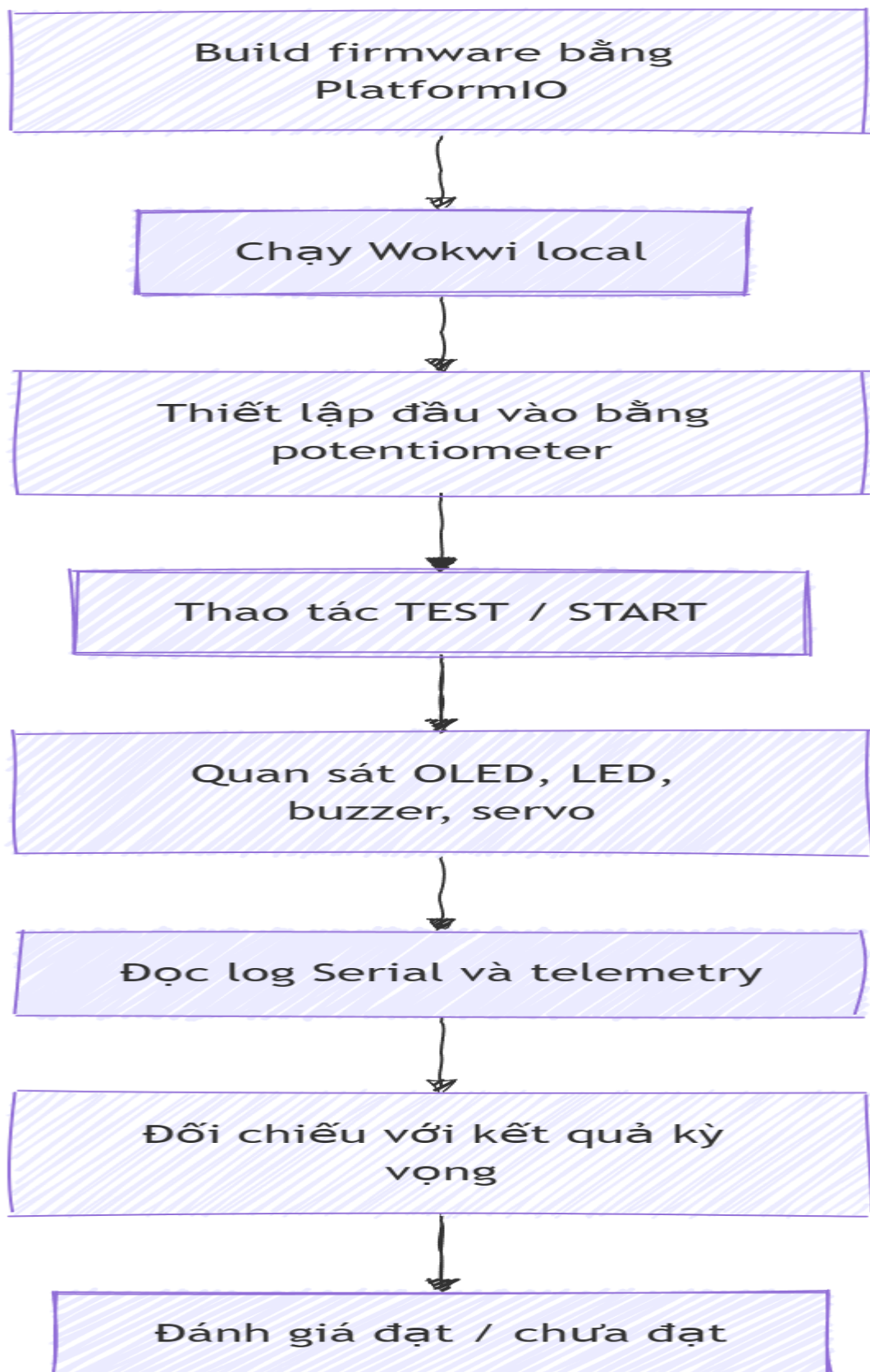
Sơ đồ mô phỏng hiện tại đã được đồng bộ với firmware ở các chân TEST = GPIO14, START = GPIO16, ADC = GPIO34, servo = GPIO15, buzzer = GPIO13 và OLED = GPIO21/22. Điều này giúp các kịch bản kiểm thử được tiến hành nhất quán giữa code và mô phỏng.

Kiểm thử theo kịch bản chức năng kết hợp quan sát log và metric, cụ thể:

- người kiểm thử thao tác các nút TEST/START và điều chỉnh giá trị đầu vào bằng potentiometer;
- quan sát trạng thái qua OLED, LED, buzzer và servo;
- đồng thời theo dõi log Serial và các bản tin telemetry để kiểm tra thời gian xử lý, số mẫu, kết quả PASS/FAIL và fault nếu có.

Đây là phương pháp phù hợp với prototype nhúng vì nó không đòi hỏi dụng cụ đo quá phức tạp, nhưng vẫn cho phép bám sát cả hành vi vật lý lẫn trạng thái logic bên trong hệ thống.

5.2.1. Sơ đồ quy trình kiểm thử đề xuất



Sơ đồ quy trình kiểm thử

5.3. Danh sách test case

5.3.1. Bảng test case đề xuất

Mã test	Tên test case	Mục tiêu	Input / thao tác	Kết quả kỳ vọng
TC01	Khởi động và preheat	Kiểm tra luồng boot	Cấp nguồn, khởi chạy mô phỏng	Hệ thống vào PREHEAT, LED vàng sáng, servo khóa, hết preheat chuyển sang STANDBY_LOCKED
TC02	PASS với đầu vào thấp	Kiểm tra nhánh đạt điều kiện	Giảm potentiometer dưới ngưỡng, nhấn TEST	Hệ thống sang SAMPLING rồi PASS_READY, buzzer pass ngắn, LED xanh
TC03	START sau PASS	Kiểm tra mở khóa đúng điều kiện	Sau TC02, nhấn START	Servo mở khóa, state sang RUNNING
TC04	FAIL với đầu vào cao	Kiểm tra nhánh không đạt	Tăng potentiometer trên ngưỡng, nhấn TEST	Hệ thống sang FAIL_LOCKED, LED đỏ, buzzer cảnh báo
TC05	START khi FAIL	Kiểm tra từ chối mở khóa	Sau TC04, nhấn START	Servo vẫn giữ khóa, không chuyển RUNNING
TC06	TEST lại sau FAIL	Kiểm tra khả năng đo lại	Sau FAIL, giảm đầu vào rồi nhấn TEST	Hệ thống cho phép sampling lại và có thể sang PASS_READY
TC07	START trong RUNNING	Kiểm tra luồng quay về standby	Sau RUNNING, nhấn START lần nữa	Hệ thống quay về STANDBY_LOCKED

TC08	Sensor warning mềm	Kiểm tra cảnh báo cảm biến	Giữ ADC gần rail trong thời gian dài nhưng chưa fault cứng	Telemetry/log cảnh báo sensor warning
TC09	SensorTimeout	Kiểm tra fault cứng	Trong suốt cửa sổ sampling, giữ ADC dính rail	Hệ thống vào ERROR_LOCKED
TC10	OLED fault	Kiểm tra lỗi khởi tạo hiển thị	Mô phỏng trường hợp OLED không khởi tạo được	Hệ thống vào ERROR_LOCKED, giữ safe state

5.3.2. Nhận xét về bộ test case

Bộ test case trên được xây dựng theo hướng vừa đủ để kiểm tra toàn bộ các nhánh logic quan trọng nhất của mô hình. Trong đó:

- TC01, TC02, TC03, TC04, TC05 là nhóm cốt lõi bắt buộc cho demo;
- TC06, TC07 kiểm tra khả năng quay lại vòng lặp vận hành;
- TC08, TC09, TC10 giúp đánh giá fault handling và sensor handling.

Cách tổ chức này phù hợp với hệ thống hiện tại vì firmware đã có đủ trạng thái, fault code và log để phản ánh những trường hợp trên.

5.4. Các chỉ số đo đạc và tiêu chí đánh giá

Mặc dù chưa có bộ kết quả thực nghiệm cuối cùng, firmware hiện tại đã hỗ trợ sẵn nhiều chỉ số có thể dùng cho đánh giá định lượng. Đây là một lợi thế lớn vì nó cho phép chuyển quá trình kiểm thử từ quan sát cảm tính sang đánh giá có cơ sở.

5.4.1. Bảng chỉ số cần đo

Mã chỉ số	Tên chỉ số	Ý nghĩa	Nguồn dữ liệu
M01	testToResultMs	Thời gian từ khi bắt đầu TEST đến khi có kết quả PASS/FAIL	Telemetry / Serial log

M02	passReadyToUnlockMs	Thời gian từ lúc hệ thống vào PASS_READY đến lúc người dùng nhấn START hợp lệ	Telemetry / Serial log
M03	startToUnlockMs	Thời gian từ lúc nhấn START đến lúc hệ thống ra lệnh mở khóa	Telemetry / Serial log
M04	sampleStdDev	Độ lệch chuẩn của cửa sổ mẫu ADC	Telemetry
M05	consecutiveFailCount	Số lần FAIL liên tiếp	Telemetry
M06	sampleCount / sampleTotal	Số mẫu đã lấy trên tổng số mẫu cần lấy	Telemetry
M07	sensorWarningDuration Ms	Thời gian duy trì cảnh báo sensor warning	Telemetry
M08	servoAngle	Góc servo hiện tại	Snapshot / telemetry

5.4.2. Tiêu chí đánh giá đạt/chưa đạt

Trong khuôn khổ hiện tại, tiêu chí đánh giá được xác định như sau:

- Một test case được xem là đạt khi:
 - trạng thái hệ thống chuyển đúng theo thiết kế;
 - OLED, LED, buzzer, servo phản hồi đúng;
 - log/telemetry khớp với hành vi quan sát được;
 - không có chuyển trạng thái sai ngoài dự kiến.
- Một test case được xem là chưa đạt khi:
 - hệ thống không chuyển đúng trạng thái mong đợi;
 - phần cứng đầu ra không khớp với trạng thái logic;
 - metric không được sinh ra hoặc bất thường;
 - hệ thống bị kẹt trạng thái hoặc bỏ qua thao tác hợp lệ.

Đánh giá của đề tài không chỉ dựa vào việc “nhìn thấy hệ thống chạy”, mà dựa trên sự thống nhất giữa ba lớp: logic bên trong, phản hồi phần cứng và dữ liệu log/telemetry.

5.5. Kết quả kiểm thử

5.5.1. Kịch bản Boot / Preheat

Nội dung kiểm tra	Kết quả kỳ vọng
Hệ thống khởi động	ESP32 khởi tạo phần cứng thành công
Trạng thái ban đầu	Servo khóa, buzzer tắt
Hiển thị ban đầu	OLED hiển thị boot/preheat
Chỉ báo	LED vàng sáng
Chuyển trạng thái	Sau kPreheatMs, hệ thống sang STANDBY_LOCKED

5.5.2. Kịch bản PASS rồi START

Nội dung kiểm tra	Kết quả kỳ vọng
Nhấn TEST với đầu vào thấp	Hệ thống vào SAMPLING
Sampling hoàn tất	Giá trị trung bình ADC nhỏ hơn threshold
Kết quả	Hệ thống sang PASS_READY

Chỉ báo	LED xanh, buzzer pass ngắn
Nhấn START	Hệ thống sang RUNNING, servo mở khóa

5.5.3. Kịch bản FAIL rồi không cho START

Nội dung kiểm tra	Kết quả kỳ vọng
Nhấn TEST với đầu vào cao	Hệ thống vào SAMPLING
Sampling hoàn tất	Giá trị trung bình ADC lớn hơn hoặc bằng threshold
Kết quả	Hệ thống sang FAIL_LOCKED
Chỉ báo	LED đỏ, buzzer fail hoạt động theo chu kỳ
Nhấn START	Không mở khóa, không sang RUNNING

5.5.4. Kịch bản SensorTimeout

Nội dung kiểm tra	Kết quả kỳ vọng
Toàn bộ mẫu dính rail thấp hoặc cao	CỜ allNearLowRail hoặc allNearHighRail giữ nguyên đến hết cửa sổ sample
Sampling hoàn tất	Hệ thống gọi enterFault(FaultCode::SensorTimeout, ...)

Kết quả	Hệ thống sang ERROR_LOCKED
Chỉ báo	Giữ khóa, hiển thị lỗi, không cho START

5.6. Hạn chế của quá trình kiểm thử hiện tại

Phần kiểm thử ở giai đoạn hiện tại còn một số hạn chế và cần được nêu rõ.

Trước hết, mô hình hiện vẫn chủ yếu dựa trên mô phỏng Wokwi, trong đó đầu vào analog được tạo bằng potentiometer thay vì MQ3 thật. Điều này có nghĩa là các kết quả PASS/FAIL hiện tại mới phản ánh đúng logic phần mềm và cơ chế lấy mẫu ADC, chứ chưa chứng minh được độ chính xác vật lý của một cảm biến đo nồng độ cồn ngoài thực tế. Vì vậy, các threshold hiện tại chỉ nên được xem là tham số demo, không phải giá trị hiệu chuẩn cuối cùng.

Thứ hai, do chưa tiến hành đầy đủ kiểm thử thực nghiệm, báo cáo hiện chưa thể đưa ra các kết luận định lượng cuối cùng như độ trễ đo được trung bình, sai số thực, độ ổn định khi chạy liên tục lâu dài hoặc mức tiêu thụ điện năng thực tế. Những nội dung này cần được bổ sung sau khi chạy test thật và thu thập log đầy đủ.

Thứ ba, môi trường Wokwi không phản ánh hoàn toàn các hiện tượng ngoài đời thật như:

- dao động nguồn khi servo tải nặng,
- nhiễu analog thực tế,
- đặc tính làm nóng của MQ3 thật,
- sai khác cơ khí của cơ cấu khóa,
- độ trễ vật lý hoàn chỉnh của servo.

Do đó, phần kiểm thử hiện tại chỉ có thể đánh giá tốt về logic điều khiển, tổ chức phần mềm và khả năng phản hồi theo thiết kế, chứ chưa thể thay thế hoàn toàn kiểm thử phần cứng thật.

CHƯƠNG 6. TÍNH SÁNG TẠO, KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Tính sáng tạo của đề tài

Mặc dù phạm vi của đề tài chỉ dừng ở mức bài tập lớn môn học, hệ thống Alcohol Interlock hiện tại không được xây dựng theo hướng “mô hình đèn nhấp nháy” đơn giản, mà có nhiều điểm thể hiện tư duy thiết kế thú vị tương đối nghiêm túc. Nhóm tập trung vào việc

kết hợp các thành phần quen thuộc thành một hệ thống có logic nghiệp vụ rõ ràng và ràng buộc an toàn cụ thể, hướng đến ứng dụng thực tế hơn là xây dựng thêm nhiều tính năng phức tạp.

Trước hết, đề tài có điểm mới ở bài toán ứng dụng. Thay vì lựa chọn các mô hình quen thuộc như điều khiển đèn, đo nhiệt độ hay cảnh báo đơn giản, hệ thống được xây dựng theo hướng mô phỏng một cơ chế khóa liên động trước khi khởi động. Điều này làm cho đề tài gắn với một tình huống thực tế rõ ràng hơn, đồng thời tạo điều kiện để áp dụng nhiều kỹ thuật nhúng hơn so với những bài toán chỉ có một luồng xử lý tuyến tính. Chính vì bài toán có tính điều kiện và có trạng thái, firmware buộc phải được tổ chức bằng FSM chứ không thể chỉ xử lý theo kiểu tuần tự đơn giản.

Điểm sáng tạo tiếp theo là cách tổ chức phần mềm theo module. Trong nhiều bài tập lớn mức cơ bản, sinh viên thường viết toàn bộ logic vào một file duy nhất, khiến chương trình khó đọc và khó mở rộng. Ở dự án này, firmware đã được tách thành các lớp chức năng rõ ràng như cấu hình, giao tiếp phần cứng, máy trạng thái, giao diện OLED và telemetry. Việc phân tầng như vậy không chỉ giúp chương trình gọn hơn, mà còn phản ánh đúng tư duy của một hệ thống nhúng có thiết kế, trong đó phần cứng, logic nghiệp vụ và hiển thị không bị trộn lẫn vào nhau. Đây là một điểm đáng ghi nhận dưới góc nhìn học thuật.

Một điểm sáng tạo khác nằm ở việc hệ thống không chỉ có PASS/FAIL đơn giản, mà còn được thiết kế với nhiều mức phản ứng trước tín hiệu bất thường. Firmware hiện phân biệt giữa sensor warning mức mềm và fault cứng SensorTimeout, thay vì gom mọi bất thường thành một trường hợp lỗi duy nhất. Cách phân tầng này cho thấy hệ thống không chỉ phản ứng theo kiểu “đo rồi so sánh”, mà còn có tư duy xử lý độ tin cậy của đầu vào. Cách tiếp cận này giúp firmware phản ứng khác nhau tùy theo mức độ bất thường của tín hiệu, thay vì gom mọi lỗi vào một trường hợp xử lý duy nhất.

Ngoài ra, đề tài còn thể hiện tính sáng tạo ở chỗ đã áp dụng một số kỹ thuật nâng cao vừa đủ để tăng chất lượng hệ thống, nhưng không làm chương trình trở nên quá phức tạp. Ví dụ:

- nút TEST được hỗ trợ bằng interrupt;
- buzzer sử dụng LEDC hardware timer;
- sampling được triển khai non-blocking bằng millis();
- telemetry được thiết kế để sinh ra metric phục vụ kiểm thử;
- mô phỏng Wokwi local được gắn trực tiếp với firmware build từ PlatformIO.

Các yếu tố này kết hợp với nhau tạo nên một hệ thống vừa đủ “thật” để có thể phân tích và đánh giá, chứ không chỉ dừng ở mức demo hình thức.

6.1.1. Bảng tổng hợp các điểm sáng tạo của đề tài

Nội dung	Mô tả
Bài toán ứng dụng	Mô phỏng cơ chế interlock an toàn trước khi khởi động
FSM rõ ràng	Hệ thống tổ chức theo máy trạng thái, không xử lý tuyến tính đơn giản
Fault handling phân tầng	Có warning mềm và fault cứng SensorTimeout
Interrupt	Sử dụng interrupt cho nút TEST
Hardware timer	Sử dụng LEDC timer cho buzzer
Telemetry định lượng	Có metric phục vụ kiểm thử và đánh giá
Mô phỏng nhất quán	Wokwi local chạy trực tiếp firmware build từ PlatformIO

6.2. Kết quả đạt được của đề tài

Dựa trên những gì đã triển khai trong repo hiện tại, có thể khẳng định rằng đề tài đã hoàn thành được các mục tiêu quan trọng ở mức prototype học phần.

Trước hết, hệ thống đã xây dựng thành công một mô hình phần cứng nhúng cơ bản hoàn chỉnh, gồm:

- ESP32 làm khối xử lý trung tâm,
- đầu vào analog mô phỏng bằng potentiometer,
- hai nút thao tác TEST và START,
- servo mô phỏng khóa/mở,
- OLED hiển thị trạng thái,
- LED và buzzer làm khối cảnh báo.

Thứ hai, firmware đã được triển khai với luồng nghiệp vụ tương đối đầy đủ cho một mô hình interlock. Hệ thống có đầy đủ các trạng thái từ preheat, standby, sampling, pass, fail, running cho tới error. Việc tổ chức logic thành máy trạng thái giúp mọi hành vi của hệ thống trở nên rõ ràng, nhất quán và dễ kiểm soát hơn.

Thứ ba, dự án đã hiện thực được một số kỹ thuật lập trình nhúng đáng chú ý như:

- debounce nút bằng phần mềm,
- sampling non-blocking,

- interrupt cho TEST,
- LEDC timer cho buzzer,
- fault handling nhiều mức,
- telemetry và log phục vụ kiểm thử.

Những kỹ thuật này cho thấy đề tài không dừng ở mức “làm cho chạy được”, mà đã chú ý đến cấu trúc, độ ổn định và khả năng đánh giá của hệ thống.

Thứ tư, source code của dự án hiện đã được tổ chức khá rõ ràng, có README mô tả hướng triển khai, có roadmap chạy mô phỏng và có cấu hình build nhất quán qua PlatformIO và Wokwi local.

6.2.1. Bảng tổng hợp kết quả đạt được

Nội dung	Mức độ đạt được
Xây dựng được mô hình nhúng hoàn chỉnh	Đạt
Thiết kế được FSM rõ ràng cho hệ thống	Đạt
Điều khiển được servo, LED, buzzer, OLED	Đạt
Xử lý được PASS / FAIL / ERROR	Đạt
Có interrupt và hardware timer	Đạt
Có log và telemetry cho kiểm thử	Đạt
Tổ chức repo và source code rõ ràng	Đạt

6.3. Hạn chế của hệ thống hiện tại

Bên cạnh những kết quả đạt được, đề tài hiện vẫn còn một số hạn chế cần được nhìn nhận rõ.

Trong phạm vi bài tập lớn, hệ thống được xác định ngay từ đầu là prototype mô phỏng chức năng. Việc chưa tích hợp cảm biến MQ3 thật và chưa kiểm thử trên phần cứng vật lý là ranh giới rõ ràng của đề tài, không phải điều nằm ngoài kế hoạch. Trong môi trường Wokwi, đầu vào analog được tạo bởi potentiometer chứ không phải cảm biến MQ3 thật, vì vậy mọi kết quả PASS/FAIL hiện tại mới chủ yếu phản ánh đúng logic xử lý và cơ chế lấy mẫu của firmware, chứ chưa thể xem là kết quả đo nồng độ cồn thực sự. Ngưỡng

kAlcoholAdc = 2000 trong code hiện mang ý nghĩa tham số demo, chưa phải ngưỡng đã hiệu chuẩn trên thiết bị thật.

Hạn chế thứ hai là cơ cấu chấp hành của hệ thống hiện chỉ ở mức servo mô phỏng khóa/mở, chưa can thiệp vào một cơ cấu khởi động thật. Điều này là phù hợp với phạm vi bài tập lớn, nhưng đồng nghĩa với việc hệ thống chưa được kiểm chứng trong bối cảnh có tải cơ khí, nguồn thực hoặc mạch relay/ignition thật. Nói cách khác, sản phẩm hiện tại chứng minh tốt về mặt logic nhúng, nhưng chưa chứng minh đầy đủ ở mức hệ cơ điện hoàn chỉnh.

Hạn chế thứ ba là phần kiểm thử thực nghiệm hiện chưa được hoàn thiện đầy đủ. Mặc dù firmware đã có sẵn metric và telemetry để phục vụ đánh giá, báo cáo hiện mới dừng ở mức kế hoạch kiểm thử, kịch bản kiểm thử và kết quả kỳ vọng. Các số liệu thực nghiệm cuối cùng như thời gian phản hồi trung bình, độ ổn định trong nhiều lần đo liên tiếp hay mức độ dao động đầu vào thực tế vẫn cần được bổ sung ở các bước tiếp theo.

Hạn chế thứ tư là hệ thống chưa đi sâu vào tối ưu năng lượng, đóng gói cơ khí, hoặc bảo vệ phần cứng. Đây là những yếu tố quan trọng nếu muốn chuyển từ mô hình học phần sang sản phẩm thực tế, nhưng hiện chưa nằm trong trọng tâm của đề tài. Vì vậy, phần hiện tại phù hợp với vai trò một prototype minh họa nguyên lý hơn là một thiết bị sẵn sàng triển khai ngoài đời.

6.3.1. Bảng tổng hợp các hạn chế hiện tại

Hạn chế	Mô tả
Đầu vào mô phỏng	Chưa dùng MQ3 thật, mới dùng potentiometer
Ngưỡng demo	Threshold hiện tại chưa phải giá trị hiệu chuẩn thực tế
Cơ cấu chấp hành đơn giản	Chưa tích hợp relay hoặc cơ cấu khởi động thật
Chưa có đầy đủ số liệu test	Chưa hoàn tất kiểm thử thực nghiệm
Chưa tối ưu năng lượng	Hệ thống chưa có chế độ tiết kiệm điện
Chưa đóng gói phần cứng	Chưa có vỏ, mạch nguồn thực hoặc giải pháp cơ khí hoàn chỉnh

6.4. Hướng phát triển của đề tài

Từ nền tảng hiện tại, hệ thống Alcohol Interlock có thể được phát triển theo nhiều hướng khác nhau nếu có thêm thời gian và điều kiện triển khai.

Hướng phát triển đầu tiên là chuyển từ mô phỏng sang phần cứng cảm biến thật. Khi đó, potentiometer sẽ được thay bằng module MQ3 hoặc một cảm biến phù hợp hơn, đồng thời hệ thống cần được hiệu chuẩn lại ngưỡng và thời gian warm-up. Đây là bước quan trọng nhất nếu muốn nâng giá trị của đề tài từ mô hình logic sang mô hình đo thực tế.

Hướng phát triển thứ hai là mở rộng cơ cấu chấp hành. Thay vì chỉ mô phỏng bằng servo, hệ thống có thể tích hợp relay hoặc mạch điều khiển tải để mô phỏng sát hơn hành vi khóa liên động trên một mạch khởi động thực tế. Đi kèm với đó là việc hoàn thiện phần nguồn, chống nhiễu và bảo vệ mạch để hệ thống vận hành ổn định hơn trên phần cứng thật.

Hướng phát triển thứ ba là nâng cao khả năng kiểm thử và giám sát. Telemetry hiện tại đã là một nền tảng tốt để mở rộng. Trong tương lai, dashboard có thể được phát triển mạnh hơn để hiển thị trực tiếp biểu đồ sample, lịch sử test, thống kê pass/fail, số lần warning hoặc số lần fault. Điều này giúp mô hình không chỉ phục vụ demo mà còn hỗ trợ phân tích trong thời gian dài.

Hướng phát triển thứ tư là tăng mức độ hoàn thiện của firmware. Ví dụ:

- bổ sung FreeRTOS task nếu cần tách sampling/UI/telemetry rõ hơn;
- thêm cơ chế lưu lịch sử hoặc cấu hình vào NVS/flash;
- thêm cơ chế cảnh báo đa mức;
- mở rộng fault handling với nhiều loại lỗi phần cứng hơn.

Những mở rộng này không bắt buộc với bài tập lớn hiện tại, nhưng là hướng phát triển tự nhiên nếu muốn nâng cấp dự án sau môn học.

Hướng phát triển cuối cùng là hoàn thiện phần trình bày và kiểm thử thực nghiệm. Khi có thêm thời gian, người thực hiện có thể chạy đầy đủ các test case, lưu log Serial, chụp ảnh mô phỏng từng trạng thái và bổ sung bảng kết quả thực tế vào Chương 5.

6.4.1. Bảng định hướng phát triển

Hướng phát triển	Mục tiêu
Thay potentiometer bằng MQ3 thật	Tăng tính thực tế của mô hình

Hiệu chuẩn threshold và warm-up	Đưa hệ thống gần hơn với vận hành thực
Tích hợp relay / tải thật	Mô phỏng sát hơn cơ chế interlock thực tế
Mở rộng dashboard	Giám sát trực quan và lưu lịch sử vận hành
Hoàn thiện fault handling	Xử lý đa dạng lỗi phần cứng hơn
Bổ sung lưu trữ cấu hình	Tăng tính hoàn chỉnh của firmware
Hoàn tất kiểm thử thực nghiệm	Bổ sung số liệu định lượng cho báo cáo
Hiệu chuẩn với cảm biến MQ3 thật	Đang tiến hành
Kiểm thử thực nghiệm đầy đủ	Đang tiến hành

KẾT LUẬN

Qua quá trình thực hiện bài tập lớn, nhóm đã xây dựng được một prototype hệ thống nhúng mô phỏng cơ chế khóa liên động kiểm tra điều kiện an toàn trước khi khởi động, chạy trên vi điều khiển ESP32 và được kiểm thử trong môi trường mô phỏng Wokwi local.

Về mặt phần cứng, hệ thống được thiết kế với đầy đủ các khối đặc trưng của một mô hình nhúng hoàn chỉnh: khối đầu vào analog và nút thao tác, khối xử lý trung tâm ESP32, khối chấp hành servo SG90, và khối phản hồi gồm OLED SSD1306, LED ba màu và buzzer. Sơ đồ kết nối đã được chốt đồng bộ giữa firmware và file mô phỏng diagram.json, sẵn sàng để lắp ráp trên phần cứng thật trong bước tiếp theo.

Về mặt phần mềm, firmware được tổ chức thành bảy module riêng biệt theo chức năng: cấu hình, giao tiếp phần cứng, điều khiển trạng thái, giao diện OLED, telemetry, kiểu dữ liệu dùng chung và entry point. Logic điều khiển được xây dựng theo mô hình máy trạng thái hữu hạn với bảy trạng thái rõ ràng, đảm bảo hành vi hệ thống nhất quán và có thể kiểm tra được. Các kỹ thuật nhúng được áp dụng bao gồm: lấy mẫu non-blocking bằng `millis()`, ngắt phần cứng với ISR tối giản cho nút TEST, điều khiển buzzer bằng LEDC hardware timer 2 kHz, debounce phần mềm và fault handling hai mức với `SensorWarning` và `FaultCode` riêng biệt.

Quá trình kiểm thử được thực hiện trên Wokwi local với ba kịch bản vận hành chính: Boot/Preheat, PASS rồi START và FAIL rồi không cho START, cùng kịch bản `SensorTimeout` khi tín hiệu đầu vào bị lỗi. Các chỉ số đo đạc từ Serial log như `test_to_result_ms`, `sampleStdDev` và `passReadyToUnlockMs` xác nhận hệ thống hoạt động đúng theo thiết kế trong môi trường mô phỏng.

Nhóm cũng nhìn nhận rõ những giới hạn còn lại của đề tài. Đầu vào hiện vẫn là potentiometer mô phỏng, chưa phải cảm biến MQ3 thật với đặc tính điện hóa thực tế. Ngưỡng `kAlcoholAdc = 2000` là tham số demo, chưa qua hiệu chuẩn vật lý. Kiểm thử trên phần cứng lắp ráp thực tế, bao gồm đo độ trễ servo, kiểm tra nhiễu nguồn và độ ổn định trong thời gian dài, chưa được thực hiện trong phạm vi bài tập lớn này.

Hướng phát triển tiếp theo mà nhóm xác định bao gồm: thay thế potentiometer bằng module MQ3 thật và hiệu chuẩn lại ngưỡng, lắp ráp và kiểm thử trên phần cứng vật lý, bổ sung relay để mô phỏng sát hơn cơ cấu khởi động thực tế, và hoàn thiện dashboard telemetry để hỗ trợ phân tích lâu dài.

Nhìn lại toàn bộ quá trình thực hiện, đề tài đã giúp nhóm hiểu sâu hơn về cách tổ chức firmware nhúng theo hướng module hóa, cách thiết kế một máy trạng thái có ràng buộc an toàn, và cách khai thác các tài nguyên phần cứng của ESP32 một cách có chủ đích. Đây là nền tảng tốt để nhóm tiếp tục phát triển hệ thống hoàn chỉnh hơn trong các bước tiếp theo.

